

**MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO**

**ARTHUR WILMER RODRIGUES XAVIER
MILENA DE SOUZA CARUBA**

**IMPLEMENTAÇÃO DE UMA CAMADA DE PROTEÇÃO BASEADA EM
ANOMALIAS NO SISTEMA DE DETECÇÃO DE INTRUSÃO (IDS) DO
MÓDULO DE PROTEÇÃO CIBERNÉTICA UTILIZANDO TÉCNICAS DE
APRENDIZAGEM DE MÁQUINA**

**RIO DE JANEIRO
2026**

ARTHUR WILMER RODRIGUES XAVIER
MILENA DE SOUZA CARUBA

IMPLEMENTAÇÃO DE UMA CAMADA DE PROTEÇÃO BASEADA EM
ANOMALIAS NO SISTEMA DE DETECÇÃO DE INTRUSÃO (IDS) DO
MÓDULO DE PROTEÇÃO CIBERNÉTICA UTILIZANDO TÉCNICAS DE
APRENDIZAGEM DE MÁQUINA

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador(es): Oscar Martins Wanderley Filho, Cap
Mateus Guimarães Lima de Freitas, 1º
Ten

Rio de Janeiro
2026

©2026

INSTITUTO MILITAR DE ENGENHARIA

Praça General Tibúrcio, 80 – Praia Vermelha

Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmar ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Xavier, Arthur Wilmer Rodrigues; Caruba, Milena de Souza.

Implementação de uma Camada de Proteção Baseada em Anomalias no Sistema de Detecção de Intrusão (IDS) do Módulo de Proteção Cibernética Utilizando Técnicas de Aprendizagem de Máquina / Arthur Wilmer Rodrigues Xavier e Milena de Souza Caruba. – Rio de Janeiro, 2026.

53 f.

Orientador(es): Oscar Martins Wanderley Filho e Mateus Guimarães Lima de Freitas.

Projeto de Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia da Computação, 2026.

1. Aprendizado de Máquina. Proteção Cibernética. Detecção de Intrusões. Mixture of Experts. i. Wanderley Filho, Oscar Martins (orient.) ii. de Freitas, Mateus Guimarães Lima (orient.) iii. Título

**ARTHUR WILMER RODRIGUES XAVIER
MILENA DE SOUZA CARUBA**

**Implementação de uma Camada de Proteção Baseada em
Anomalias no Sistema de Detecção de Intrusão (IDS) do
Módulo de Proteção Cibernética Utilizando Técnicas de
Aprendizagem de Máquina**

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador(es): Oscar Martins Wanderley Filho e Mateus Guimarães Lima de Freitas.

Aprovada em 8 de outubro de 2026, pela seguinte banca examinadora:

Prof. **Oscar Martins Wanderley Filho** - M.Sc. do IME - Presidente

Prof. **Mateus Guimarães Lima de Freitas** - 1º Ten do CDS

Prof. **Leandro de Mattos Ferreira** - M.Sc do IME

Prof. **Anderson Fernandes Pereira dos Santos** - D.Sc. do IME

Rio de Janeiro
2026

RESUMO

Este trabalho tem como objetivo dar continuidade à evolução do Módulo de Proteção Cibernética por meio de uma maior automação do provisionamento de seu ambiente virtualizado e da prova de conceito do Oráculo, módulo ao qual está integrado um modelo baseado na arquitetura Mixture of Experts (MoE), em comparação com o Suricata, ferramenta de detecção por assinaturas atualmente utilizada. A proposta parte da limitação dos mecanismos tradicionais de detecção baseados em assinaturas, que dependem de regras previamente cadastradas e, por isso, podem apresentar menor efetividade diante de ameaças desconhecidas ou de variações de ataques já existentes. Nesse contexto, investiga-se o uso do Oráculo como uma camada complementar de detecção de anomalias em redes privadas. O projeto contempla a implementação de um script que automatize a implantação do ambiente MPC 1.0 virtualizado sem o uso de virtualização aninhada, a elaboração de um manual de usuário para sua replicação nas Organizações Militares, a calibração do método de comparação entre o Oráculo e o Suricata, e a posterior comparação experimental entre o modelo MoE recalibrado e o Suricata, considerando métricas de classificação, latência e consumo de recursos. A finalidade é viabilizar a implantação funcional desse tipo de rede neural em ambiente embarcado, com foco na detecção de intrusões em redes privadas. A presente versão do trabalho apresenta a construção do ambiente MPC 1.0 virtualizado, o desenvolvimento do script de automação para sua implantação, a validação do fluxo básico de conectividade e do espelhamento de tráfego, e a calibração do método de comparação entre o Oráculo e o Suricata, obtida por meio da investigação metodológica dos resultados experimentais de trabalhos anteriores e testada empiricamente pela replicação do experimento original e por um segundo experimento com um novo conjunto de regras, elaborado especificamente para o tráfego avaliado.

Palavras-chave: Aprendizado de Máquina. Proteção Cibernética. Detecção de Intrusões. Mixture of Experts.

ABSTRACT

This work aims to advance the evolution of the Cyber Protection Module (MPC 1.0), through the automation of its virtualized environment provisioning and the proof of concept of the Oracle, a module integrated with a model based on the Mixture of Experts (MoE) architecture, in comparison with Suricata, the signature-based detection tool currently in use. The proposal is motivated by the limitations of traditional signature-based detection mechanisms, which rely on previously registered rules and, therefore, may be less effective against unknown threats or variations of existing attacks. In this context, the use of the Oracle is investigated as a complementary anomaly detection layer in private networks. The project includes the implementation of a script that automates the deployment of the virtualized MPC 1.0 environment without nested virtualization, the preparation of a user manual for its replication across Military Organizations, the calibration of the comparison method between the Oracle and Suricata, and the subsequent experimental comparison between the recalibrated MoE model and Suricata, considering classification metrics, latency, and resource consumption. The purpose is to enable the functional deployment of this type of neural network in an embedded environment, with a focus on intrusion detection in private networks. This version of the work presents the construction of the virtualized MPC 1.0 environment, the development of the automation script for its deployment, the validation of the basic connectivity flow and traffic mirroring, and the calibration of the comparison method between the Oracle and Suricata, obtained through the methodological investigation of experimental results from previous work and empirically tested through replication of the original experiment and a second experiment using a new rule set tailored to the traffic under evaluation.

Keywords: Machine Learning. Cyber Protection. Intrusion Detection. Mixture of Experts.

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama de rede do Nó de Acesso ligada com a malha nodal, com a rede local e com a EBNet	18
Figura 2 – Topologia detalhada da infraestrutura original.	28
Figura 3 – Fluxograma que descreve o provisionamento do ambiente virtual do MPC 1.0	30
Figura 4 – Camadas funcionais do ambiente de testes.	35
Figura 5 – Cronograma do projeto	47

LISTA DE TABELAS

Tabela 1	–	Configurações das VMs provisionadas no <i>Proxmox</i>	32
Tabela 2	–	Interfaces da VM 100 (pfSense CE 2.7.2).	33
Tabela 3	–	Interfaces da VM 101 (Security Onion 2.3.300).	33
Tabela 4	–	Resultados da replicação do experimento original.	45
Tabela 5	–	Métricas do Suricata recalculadas a partir da contagem nativa de fluxos.	46
Tabela 6	–	Resultados obtidos com o novo conjunto de regras.	46
Tabela 7	–	Métricas do Suricata com o novo conjunto de regras.	46

LISTA DE ABREVIATURAS E SIGLAS

IDS	Intrusion Detection System (Sistema de Detecção de Intrusões)
IPS	Intrusion Prevention System (Sistema de Prevenção de Intrusões)
MoE	Mixture of Experts (Mistura de Especialistas)
MPC	Módulo de Proteção Cibernética
NIDS	Network-based Intrusion Detection Systems (Sistemas de Detecção de Intrusões baseados em Rede)
pfSense	Packet Filter Sense (Firewall baseado em FreeBSD)
Proxmox VE	Proxmox Virtual Environment (Ambiente Virtual Proxmox)

SUMÁRIO

1	INTRODUÇÃO	11
1.1	MOTIVAÇÃO	11
1.2	CARACTERIZAÇÃO DO PROBLEMA	12
1.3	OBJETIVO	13
1.4	METODOLOGIA	13
2	REFERENCIAL TEÓRICO	15
2.1	SISTEMAS DE DETECÇÃO DE INTRUSÕES (IDS)	15
2.2	INFRAESTRUTURA DE MONITORAMENTO: SECURITY UNION E PF-SENSE	16
2.3	MÓDULO DE PROTEÇÃO CIBERNÉTICA	17
2.4	ARQUITETURAS DE APRENDIZAGEM DE MÁQUINA PARA DETECÇÃO DE INTRUSÃO	18
2.4.1	REDES NEURAIIS FEEDFORWARD	18
2.4.2	REDES NEURAIIS CONVOLUCIONAIS	19
2.4.3	REDES LSTM (LONG SHORT-TERM MEMORY)	20
2.4.4	MECANISMOS DE ATENÇÃO	20
2.4.5	MIXTURE OF EXPERTS	21
2.5	MÉTRICAS DE AVALIAÇÃO DE DESEMPENHO	21
2.6	FERRAMENTAS DO AMBIENTE DE TESTE	23
2.6.1	CICFLOWMETER	23
2.6.2	RABBITMQ E O PROTOCOLO AMQP	23
2.6.3	OBSERVABILIDADE: MÉTRICAS E LOGS	24
3	TRABALHOS ANTERIORES	25
3.1	IMPACTOS DO PRÉ-PROCESSAMENTO DE DADOS E DA OTIMIZAÇÃO DE HIPERPARÂMETROS NO DESEMPENHO DE MODELOS DE APRENDIZADO DE MÁQUINA APLICADOS A SISTEMAS DE DETECÇÃO DE INTRUSÃO	25
3.2	IMPLEMENTAÇÃO DE UMA CAMADA DE PROTEÇÃO BASEADA EM ANOMALIAS NO SISTEMA DE DETECÇÃO DE INTRUSÃO DO MÓDULO DE PROTEÇÃO CIBERNÉTICA UTILIZANDO TÉCNICAS DE APRENDIZADO DE MÁQUINA	26
3.2.1	PROJETO FINAL DE CURSO 2024	26
3.2.2	PROJETO FINAL DE CURSO 2025	26

4	METODOLOGIA	28
4.1	VISÃO GERAL DA ARQUITETURA	28
4.2	VIRTUALIZAÇÃO DO MPC 1.0	29
4.2.1	AUTOMAÇÃO DO PROVISIONAMENTO	29
4.2.2	FLUXO DE DADOS	31
4.2.3	ESPECIFICAÇÕES DAS MÁQUINAS VIRTUAIS	31
4.2.3.1	PAPÉIS DOS NÓS DO AMBIENTE	32
4.2.4	CONFIGURAÇÕES DE INTERFACES DE REDE	32
4.2.5	VALIDAÇÃO DO AMBIENTE	33
4.3	AMBIENTE DE TESTE	34
4.3.1	VISÃO GERAL DO PIPELINE	34
4.3.2	INGESTÃO E PRÉ-PROCESSAMENTO	35
4.3.3	CAMADA DE CLASSIFICAÇÃO	36
4.3.4	CAMADA DE DETECÇÃO POR ASSINATURAS	37
4.3.5	CAMADA DE OBSERVABILIDADE	38
4.4	INVESTIGAÇÃO DO PIPELINE	39
4.4.1	DEFINIÇÃO DE FLUXO: CICFLOWMETER VS. SURICATA	39
4.4.2	CONTAGEM DE FLUXOS E ALERTAS	40
4.4.3	O DATASET CICIOMT2024	40
4.4.4	CONJUNTO DE REGRAS UTILIZADO	41
5	EXPERIMENTO	43
5.1	REPLICAÇÃO DO EXPERIMENTO ORIGINAL	43
5.2	EXPERIMENTO COM NOVO CONJUNTO DE REGRAS	43
6	RESULTADOS	45
6.1	RESULTADOS DA REPLICAÇÃO	45
6.2	RESULTADOS COM O NOVO CONJUNTO DE REGRAS	46
7	CRONOGRAMA	47
8	CONCLUSÃO	49
	REFERÊNCIAS	51

1 INTRODUÇÃO

O atual cenário de transformação digital, marcado pelo avanço da conectividade e pela integração de sistemas computacionais, ao mesmo tempo em que potencializa ganhos de eficiência e produtividade, expõe ativos digitais a um conjunto crescente de ameaças (1). Esse risco é particularmente sensível em ambientes de rede, onde o crescimento exponencial do volume e da heterogeneidade do tráfego têm imposto desafios significativos aos mecanismos de segurança cibernética, sobretudo no que se refere à detecção de ameaças em tempo real (2).

Diante dessa realidade, abordagens convencionais que dependem de padrões previamente catalogados conseguem reconhecer apenas ataques já registrados e mostram-se pouco eficazes contra ameaças inéditas. Como resposta a essas limitações, técnicas de aprendizagem de máquina baseadas em redes neurais profundas vêm ganhando protagonismo por permitirem a identificação automática de comportamentos anômalos no tráfego sem que seja necessário conhecimento prévio sobre cada tipo de ameaça (3).

No domínio militar, onde as redes operacionais de Comando e Controle (C2) sustentam a consciência situacional e a efetividade tática, a importância dessas técnicas torna-se ainda mais evidente (4). A indisponibilidade ou o comprometimento dessa infraestrutura pode acarretar prejuízos estratégicos significativos, tornando imprescindível o desenvolvimento de soluções defensivas robustas e autônomas, capazes de operar mesmo sob severas restrições de recursos.

1.1 Motivação

No Exército Brasileiro, o Sistema de Comando e Controle da Força Terrestre (SC2FTer) sustenta-se sobre um conjunto de redes operacionais formadas a partir dos Nós de Acesso (NA) das Organizações Militares de Comunicações. Essas redes carregam o tráfego operacional crítico, viabilizando a transmissão de ordens, o compartilhamento de dados de inteligência e a coordenação entre os elementos militares envolvidos em uma operação e, por isso, constituem alvos prioritários em cenários de conflito cibernético. Ataques bem-sucedidos contra equipamentos centrais dessa infraestrutura podem causar o comprometimento dos serviços de Comando e Controle e da consciência situacional em todos os níveis.

Foi diante dessa preocupação que se concebeu o Módulo de Proteção Cibernética (MPC), solução de hardware e software desenvolvida pelo Exército Brasileiro com o propósito de proteger as redes operacionais nas quais opera o SC2FTer (5). O MPC

desempenha funções de Monitoramento de Segurança de Rede (NSM), permitindo a detecção de eventos suspeitos, a coleta de telemetria e a investigação de incidentes nos pontos críticos da infraestrutura. Sua implantação representa um passo importante na consolidação de uma capacidade defensiva nacional, alinhada às necessidades específicas das redes táticas do Exército.

Apesar de já desempenhar um papel relevante na proteção dessas redes, o MPC opera, em sua versão atual, com mecanismos majoritariamente apoiados em conhecimento prévio sobre as ameaças. Como discutido anteriormente, esse paradigma mostra-se insuficiente diante de ataques que se afastam dos padrões já registrados, especialmente em ambientes operacionais sujeitos a condições adversas e a adversários que adaptam suas táticas ao contexto de cada operação. Essa limitação revela uma oportunidade clara de evolução do produto, motivando a investigação de mecanismos complementares de detecção capazes de identificar comportamentos anômalos sem dependência de assinaturas previamente catalogadas.

1.2 Caracterização do problema

Em sua versão atual, o MPC apoia-se no Suricata como sensor principal de detecção de intrusões (5). Essa ferramenta opera segundo o paradigma de detecção baseada em assinaturas, no qual cada pacote ou fluxo de rede é comparado com um conjunto de regras previamente cadastradas que descrevem padrões maliciosos conhecidos. Essa abordagem é eficaz contra ameaças catalogadas e amplamente difundidas, mas, conforme já apontado, mostra-se limitada diante de ataques que se afastam dos padrões registrados. A limitação é particularmente sentida em redes táticas, que operam de forma autônoma e podem permanecer longos períodos sem acesso a atualizações de regras.

Trabalhos anteriores conduzidos no contexto do MPC iniciaram a transição para uma arquitetura mais adaptativa, baseada na detecção de anomalias por modelos de aprendizado de máquina. Lima et al. (6) estabeleceram uma base experimental para o uso desses modelos no contexto do produto. Posteriormente, Santos e Cuadros (7) propuseram uma camada de detecção comportamental encapsulada em uma API denominada Oráculo, adotando a Mistura de Especialistas (Mixture of Experts, MoE) como arquitetura para o modelo de detecção. Em sequência, Asfora e Soares (8) integraram o Oráculo a um ambiente virtualizado do MPC e conduziram a primeira comparação experimental entre o modelo MoE e o Suricata, evidenciando a viabilidade técnica da arquitetura proposta.

Apesar dos avanços alcançados, dois pontos permaneceram em aberto ao final desse percurso. No plano da infraestrutura, o ambiente virtualizado do MPC foi construído sobre virtualização aninhada (nested), configuração que impõe limitações de desempenho, eleva a complexidade do provisionamento e compromete a estabilidade da execução. No plano

da detecção, as comparações entre o Oráculo e o Suricata não obtiveram resultados tão coerentes, o que indicou possíveis falhas nas métricas e nos métodos utilizados. O presente trabalho atua sobre essas duas frentes, propondo um pipeline de automação capaz de provisionar o MPC virtualizado sem recorrer à virtualização aninhada e de realizar uma comparação adequada entre Suricata e o MoE.

1.3 Objetivo

Este trabalho tem como objetivo geral dar continuidade à evolução do Módulo de Proteção Cibernética, MPC 1.0, por meio de duas frentes: a automação do provisionamento de seu ambiente virtualizado e a prova de conceito do Oráculo, módulo baseado na arquitetura Mixture of Experts (MoE), em comparação com o Suricata, ferramenta de detecção atualmente utilizada. A finalidade é viabilizar a implantação funcional desse tipo de rede neural em ambiente embarcado, com foco na detecção de intrusões em redes privadas.

Os objetivos específicos do projeto são:

- (i) Implementar um script que automatize a implantação do sistema MPC 1.0 virtualizado, sem a utilização de virtualização aninhada (nested);
- (ii) Elaborar um manual de usuário que oriente a implantação do ambiente nas diversas Organizações Militares, para treinamento e capacitação de pessoal;
- (iii) Realizar a calibração do método de comparação entre o MoE e o Suricata;
- (iv) Comparar experimentalmente o desempenho do modelo MoE recalibrado com o do Suricata, considerando métricas de classificação, latência e consumo de recursos.

1.4 Metodologia

A metodologia adotada neste trabalho organiza-se em quatro frentes complementares, que cobrem desde a familiarização com o estado atual do produto até a validação experimental dos resultados.

O trabalho iniciou-se com o estudo teórico do MPC 1.0 e a análise do ambiente virtualizado herdado do projeto anterior. Esse ambiente, no entanto, não pôde ser reaproveitado, pois fora construído sobre virtualização aninhada, configuração que impõe limitações de desempenho e eleva a complexidade do provisionamento. A análise focou, portanto, em compreender o funcionamento dos componentes em operação no MPC e a estrutura do ambiente original.

Em seguida, partiu-se para o desenvolvimento da infraestrutura, com a construção de um novo ambiente virtualizado para o MPC 1.0, sem a utilização de virtualização aninhada. Sobre esse ambiente, foi desenvolvido um *script* de automação responsável por provisioná-lo de forma reproduzível, acompanhado da elaboração do manual e do vídeo de instrução destinados a apoiar sua replicação nas Organizações Militares, para capacitação de pessoal no uso do MPC.

A etapa seguinte foi dedicada ao estudo teórico do modelo Mistura de Especialistas (MoE), à investigação da pipeline aplicada pelo Oráculo e ao entendimento dos problemas encontrados na comparação entre o Suricata e o MoE nos trabalhos anteriores. Essa investigação buscou diagnosticar as causas dos problemas observados nas comparações em trabalhos anteriores. Em seguida, será padronizado o fluxo de dados entregue ao Suricata e ao Oráculo, buscando garantir que, no futuro, ambos os sistemas operem sobre amostras equivalentes nos testes subsequentes.

Desse modo, entende-se que a análise comparativa de desempenho entre o Oráculo e o Suricata marcará o encerramento do trabalho.

2 REFERENCIAL TEÓRICO

2.1 Sistemas de Detecção de Intrusões (IDS)

A proteção de redes contra ameaças cibernéticas depende, em grande medida, da capacidade de detectar incidentes de segurança por meio do monitoramento dos eventos de rede (9). Nesse contexto, os Sistemas de Detecção de Intrusões (*Intrusion Detection Systems*, IDS) figuram entre as tecnologias mais consolidadas, sendo responsáveis por identificar comportamentos suspeitos, tentativas de invasão ou violações de políticas de segurança em tráfego de rede ou atividade de sistemas computacionais (10). Sua função não é bloquear o ataque, mas sim identificar sua ocorrência para que medidas de resposta possam ser acionadas, contribuindo assim para a preservação da confidencialidade, integridade e disponibilidade diante de ameaças que conseguem contornar mecanismos preventivos (10).

Segundo Martins et al. (11), os IDS são tradicionalmente classificados segundo dois critérios principais: a fonte dos dados que monitoram e o método empregado para identificar comportamentos maliciosos. Quanto à fonte, distinguem-se duas categorias principais: os IDS baseados em *host* (*Host-based Intrusion Detection Systems*, HIDS) e os IDS baseados em rede (*Network-based Intrusion Detection Systems*, NIDS). Os HIDS operam sobre máquinas individuais, monitorando elementos como modificações em arquivos, uso de memória, registros de auditoria e logs do sistema, com o propósito de identificar se aquela máquina específica foi comprometida. Já os NIDS atuam sobre o tráfego que circula pelas redes, analisando pacotes em busca de comportamentos suspeitos sem expor o sistema monitorado às potenciais ameaças. Soluções consolidadas exemplificam o uso de cada classe: OSSEC e Tripwire no caso dos HIDS, e Snort e Suricata no caso dos NIDS.

Quanto ao método de detecção, os IDS dividem-se entre os baseados em assinaturas (*Signature-based Intrusion Detection Systems*, SIDS) e os baseados em anomalias (*Anomaly-based Intrusion Detection Systems*, AIDS) (10, 3). A detecção por assinaturas funciona pela comparação direta entre o tráfego ou a atividade observada e um conjunto de regras previamente cadastradas, cada uma descrevendo um padrão característico de algum ataque conhecido. Já a detecção por anomalias adota uma lógica distinta: durante uma fase de treinamento, esses sistemas constroem um modelo do que constitui o comportamento normal do ambiente monitorado e, em operação, qualquer atividade que se desvie significativamente desse modelo é sinalizada como suspeita (10).

Cada uma dessas abordagens carrega vantagens e limitações próprias. A detecção por assinaturas é precisa contra ameaças catalogadas e tende a apresentar baixa taxa de

falsos positivos, mas é estruturalmente incapaz de identificar ataques que não correspondam a nenhuma assinatura previamente registrada (10, 11). A detecção por anomalias, por sua vez, oferece a capacidade de identificar ameaças desconhecidas sem dependência de catálogo prévio, mas tende a apresentar taxas mais elevadas de falsos positivos, uma vez que a fronteira entre comportamento legítimo e atípico nem sempre é facilmente delimitada (11). Esses perfis complementares sugerem que cada método cobre um espectro distinto de ameaças, abrindo espaço para que ambos sejam empregados em conjunto em uma mesma arquitetura defensiva.

2.2 Infraestrutura de Monitoramento: Security Onion e PfSense

O Security Onion é uma plataforma de código aberto voltada à implementação prática do Monitoramento de Segurança de Rede, fornecendo um ambiente integrado para coleta, análise e visualização de eventos relacionados à segurança em uma infraestrutura computacional (12). Em vez de ser uma ferramenta única dedicada a uma função específica, a plataforma reúne sob uma mesma distribuição um conjunto de soluções consolidadas, organizadas em torno de três grupos principais de componentes: o sensor de detecção Suricata, o analisador comportamental Zeek, e o conjunto Elasticsearch, Logstash e Kibana (ELK) (12). A coordenação entre esses componentes oferece uma visão consolidada do ambiente monitorado, atendendo simultaneamente a necessidades de detecção em tempo real e de investigação retrospectiva.

Entre os componentes do Security Onion, o Suricata ocupa posição central como sensor de detecção de intrusões em rede (12). Trata-se de um NIDS de código aberto que opera segundo o paradigma de detecção por assinaturas, comparando o tráfego observado com um conjunto configurável de regras que descrevem padrões de ataques conhecidos (9). Além das regras pré-definidas distribuídas com a ferramenta, o Suricata permite a definição de regras personalizadas, o que confere flexibilidade para adaptar a detecção a contextos operacionais específicos ou a ameaças particulares ao ambiente monitorado (12).

O pfSense é um *firewall* e roteador de código aberto, baseado no sistema operacional FreeBSD e mantido pela Netgate, empresa norte-americana especializada em soluções de segurança e conectividade de rede (13). A ferramenta reúne em uma única plataforma funções tradicionalmente associadas a equipamentos dedicados de borda, como filtragem de pacotes, tradução de endereços de rede (NAT), roteamento e gerenciamento de redes virtuais privadas (VPN), administradas por meio de uma interface web simplificada. A solução é utilizada em implantações de escalas variadas, de redes domésticas de pequeno porte a infraestruturas corporativas e acadêmicas de grande escala, sendo compatível tanto com dispositivos de hardware reduzido quanto com servidores de maior capacidade de processamento (13).

2.3 Módulo de Proteção Cibernética

O Módulo de Proteção Cibernética (MPC) é uma solução de hardware e software desenvolvida pelo Exército Brasileiro para o robustecimento da capacidade defensiva das redes operacionais do Sistema de Comando e Controle da Força Terrestre (SC2FTer) (5). Sua finalidade é apoiar a Proteção Cibernética Tática das Organizações Militares de Comunicações, com aplicação prioritária nas redes de telemática formadas a partir dos Nós de Acesso (NA), atuando na mitigação de ataques cibernéticos, no monitoramento de vulnerabilidades e na redução do risco de interceptação de tráfego. Ao posicionar-se como uma camada dedicada de defesa nesses pontos críticos da infraestrutura, o MPC contribui para a continuidade operacional dos serviços de Comando e Controle em ambientes táticos.

Em termos arquiteturais, o MPC implementa o paradigma de monitoramento contínuo discutido na Seção 2.1, consolidando funções de detecção de eventos suspeitos, coleta de telemetria e investigação de incidentes em uma única infraestrutura. Sua infraestrutura é construída sobre o Proxmox VE, plataforma de virtualização de código aberto baseada em Debian GNU/Linux que combina as tecnologias *Kernel-based Virtual Machine* (KVM) e *Linux Containers* (LXC) para gerenciar máquinas virtuais e contêineres a partir de uma interface web unificada (14). Nesse ambiente, o pfSense é instanciado em uma máquina virtual dedicada ao controle de perímetro, definindo a fronteira lógica do ambiente protegido, enquanto o Security Onion ocupa outra máquina virtual e atua como plataforma central de monitoramento, agregando os dados coletados pelos sensores e disponibilizando recursos para análise e investigação de eventos. Esses serviços são instanciados em ambientes isolados, garantindo modularidade, segurança e facilidade de manutenção

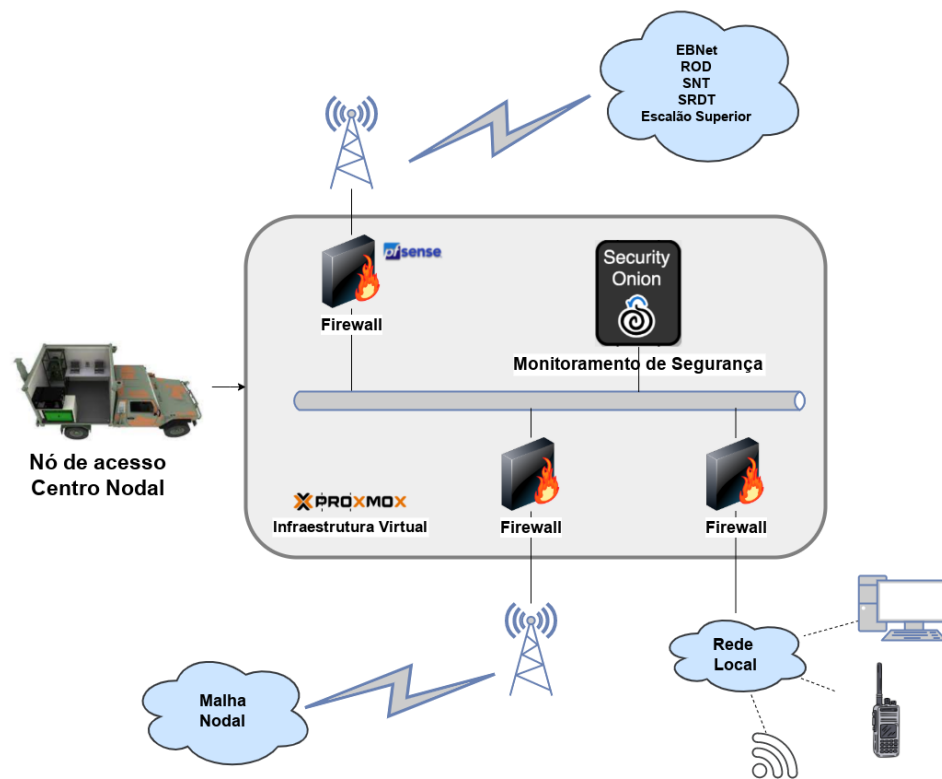


Figura 1 – Diagrama de rede do Nó de Acesso ligada com a malha nodal, com a rede local e com a EBNet

2.4 Arquiteturas de Aprendizagem de Máquina para Detecção de Intrusão

Ao contrário dos mecanismos baseados em regras e assinaturas discutidos anteriormente, modelos de aprendizado de máquina e aprendizado profundo não exigem a definição manual de quais atributos do tráfego são relevantes: aprendem tais representações diretamente a partir dos dados, em um processo de treinamento único que combina extração de atributos e classificação (3). As subseções a seguir apresentam as arquiteturas de redes neurais que compõem o modelo de classificação utilizado no âmbito deste trabalho: feedforward, convolucionais, recorrentes e baseadas em atenção, encerrando com a arquitetura Mixture of Experts, responsável por coordenar esses componentes por meio de um esquema de roteamento especializado.

2.4.1 Redes Neurais Feedforward

As Redes Neurais Feedforward, também chamadas de perceptrons multicamadas (MLPs), processam a informação em um único sentido, do vetor de entrada até a camada de saída, sem laços de realimentação entre neurônios. Cada neurônio de uma camada oculta recebe as saídas da camada anterior, calcula uma soma ponderada desses valores e aplica sobre o resultado uma função de ativação não linear, cuja saída é então repassada à

camada seguinte (15). Segundo Hornik (16), uma rede com apenas uma camada oculta é capaz de atuar como um aproximador universal, isto é, de representar qualquer função contínua com a precisão desejada, desde que utilize uma função de ativação limitada e não constante e possua um número suficiente de unidades ocultas. Entretanto, na prática, esse número de unidades necessário para aproximar funções complexas pode ser inviavelmente grande, o que motiva a preferência por redes com múltiplas camadas ocultas mais estreitas (15).

O aumento da profundidade de uma rede feedforward, no entanto, não implica necessariamente melhor desempenho pois redes mais profundas tendem a se tornar mais difíceis de treinar. O ajuste dos pesos de uma rede neural é feito por retropropagação, algoritmo que calcula o gradiente do erro em relação a cada peso percorrendo a rede da saída para a entrada, camada por camada. Em redes muito profundas, esse gradiente tende a diminuir progressivamente a cada camada percorrida, fenômeno conhecido como degradação do gradiente (17), de modo que as camadas mais próximas da entrada recebem um sinal de ajuste tão pequeno que praticamente deixam de aprender.

Para contornar esse tipo de problema, He et al. propuseram as conexões residuais. Nessa abordagem, um bloco de camadas não recebe apenas a tarefa de transformar sua entrada na saída desejada: sua saída final é obtida somando, elemento a elemento, o resultado processado pelas camadas à própria entrada original do bloco, que segue adiante por um caminho paralelo sem passar por nenhuma transformação. Assim, em vez de aprender a reconstruir toda a informação de entrada nas camadas seguintes, o bloco aprende apenas o ajuste, ou resíduo, que deve ser somado a essa entrada para chegar ao resultado desejado (18). Esse mecanismo permite o treinamento efetivo de redes com um número muito maior de camadas.

2.4.2 Redes Neurais Convolucionais

As Redes Neurais Convolucionais (CNNs) diferem das redes feedforward densamente conectadas por não ligarem cada neurônio a todas as saídas da camada anterior. Em vez disso, cada neurônio de uma camada convolucional recebe entrada apenas de uma pequena região local dos dados, também conhecido como campo receptivo, e o mesmo conjunto de pesos, organizado na forma de um filtro, é reutilizado ao percorrer toda a extensão da entrada, produzindo um mapa de características (15). Esse compartilhamento de pesos reduz drasticamente o número de parâmetros da rede em comparação a uma camada totalmente conectada que recebesse a mesma entrada, e faz com que o filtro aprenda a reconhecer um mesmo padrão independentemente de sua posição na entrada.

Embora sejam mais comumente associadas a dados bidimensionais, como imagens, as operações de convolução também podem ser aplicadas a dados de uma única dimensão, como séries temporais ou vetores de atributos organizados sequencialmente, por meio

de camadas convolucionais unidimensionais. Nesse caso, o eixo percorrido pelo filtro corresponde à progressão sequencial dos dados. Por padrão, a operação de convolução reduz a dimensão da saída em relação à entrada, já que o filtro só pode ser posicionado onde cabe inteiramente dentro dos dados: quanto maior o filtro, maior essa redução. Para evitar esse encolhimento, é possível adicionar *padding*, isto é, valores nulos inseridos artificialmente nas bordas da entrada, em quantidade suficiente para que a saída resultante tenha o mesmo tamanho da entrada original, prática conhecida como convolução *same* (15).

2.4.3 Redes LSTM (Long Short-Term Memory)

As redes Long Short-Term Memory (LSTM) pertencem à família das redes neurais recorrentes, caracterizadas por processar dados sequencialmente, um elemento por vez, mantendo a cada passo um vetor de estado, chamado de estado oculto, que resume as informações relevantes observadas até aquele ponto da sequência (15). As LSTMs foram propostas para lidar com dependências de longo prazo em dados sequenciais, superando a limitação das redes recorrentes tradicionais, que tendem a perder a capacidade de propagar informações relevantes ao longo de sequências extensas (19). Para isso, a LSTM mantém, além desse estado oculto, um estado de célula que atravessa a sequência com alterações controladas, funcionando como uma via pela qual informações podem ser preservadas por muitos passos de tempo sem serem progressivamente atenuadas.

Em sua formulação consolidada, o fluxo de informações da LSTM é regulado por três portões: o portão de entrada, que determina quanto da nova informação recebida deve ser incorporada ao estado de célula; o portão de esquecimento, que determina quanto da informação previamente armazenada deve ser descartada; e o portão de saída, que determina quanto do estado de célula deve ser exposto como saída da unidade naquele passo de tempo. Cada portão é implementado como uma camada com ativação sigmoide, produzindo valores entre 0 e 1 que ponderam a passagem de informação. A atuação conjunta desses três mecanismos é o que permite à LSTM reter seletivamente informações relevantes por muitos passos de tempo, sem que sejam sobrescritas ou atenuadas a cada nova entrada processada(15).

2.4.4 Mecanismos de Atenção

Os mecanismos de atenção permitem que um modelo pondere a relevância de diferentes posições de uma sequência ao computar a representação de cada elemento, em vez de depender exclusivamente da ordem sequencial de processamento característica das redes recorrentes. Para isso, cada elemento da sequência é projetado em três vetores, chamados de consulta, chave e valor, e a saída de cada posição é calculada como uma soma ponderada dos valores de toda a sequência, em que o peso atribuído a cada valor

depende de quão compatível sua chave é com a consulta da posição sendo processada (20). Quando esses três vetores são extraídos da própria sequência de entrada, em vez de virem de uma sequência externa, o mecanismo é chamado de auto-atenção (20). É possível ainda repetir esse processo em paralelo, com múltiplas projeções aprendidas ao mesmo tempo, de modo que o modelo capture diferentes tipos de relação entre as posições da sequência simultaneamente, sendo essa variante conhecida como *multi-head attention* (20).

2.4.5 Mixture of Experts

A arquitetura Mixture of Experts (MoE) parte de uma premissa diferente das arquiteturas apresentadas até aqui. Em vez de um único modelo processar toda a entrada, o problema é dividido entre múltiplas redes menores, cada uma especializada em lidar com um subconjunto dos casos de treinamento (21). Essa divisão é coordenada por uma rede de roteamento, que recebe a mesma entrada fornecida às redes especialistas e aprende a estimar, para cada uma delas, o quanto sua saída deve ser considerada na resposta final do sistema.

Uma motivação adicional para o uso de arquiteturas MoE está na relação entre capacidade do modelo e custo computacional. Shazeer et al. observaram que a capacidade de uma rede neural de absorver informação é limitada pelo seu número de parâmetros, e que simplesmente aumentar esse número eleva proporcionalmente o custo computacional de cada inferência. Como alternativa, propuseram um mecanismo de computação condicional, no qual apenas uma combinação esparsa dos especialistas disponíveis é ativada para cada exemplo processado, permitindo aumentar a capacidade total do modelo sem que o custo computacional por inferência cresça na mesma proporção (22).

Cada especialista, nesse esquema, é por si só uma rede neural, podendo assumir, em princípio, qualquer arquitetura, desde que sua entrada e saída sejam compatíveis com as das demais. A saída final do sistema é obtida somando as respostas dos especialistas selecionados, ponderadas pelo peso que a rede de roteamento atribui a cada um deles, de modo que especialistas considerados mais relevantes para aquela entrada específica exerçam maior influência sobre a decisão final (22).

2.5 Métricas de Avaliação de Desempenho

Esta seção apresenta as métricas empregadas por Asfora e Soares (2025) na avaliação comparativa entre o Oráculo e o Suricata, retomadas aqui para subsidiar a interpretação dos resultados discutidos na Seção 4.4. Em ambos os sistemas, o problema é tratado como uma classificação binária, ou seja, cada amostra de tráfego é rotulada como maliciosa (classe positiva) ou benigna (classe negativa), independentemente do tipo específico de ataque identificado.

A avaliação de um classificador binário parte da matriz de confusão, que organiza os resultados de classificação em quatro categorias, conforme a predição do modelo é comparada ao rótulo real de cada amostra:

- Verdadeiros Positivos (TP): amostras maliciosas corretamente identificadas como ataque;
- Falsos Negativos (FN): amostras maliciosas não detectadas pelo modelo;
- Verdadeiros Negativos (TN): amostras benignas corretamente identificadas como tráfego legítimo;
- Falsos Positivos (FP): amostras benignas incorretamente classificadas como ataque.

A partir desses quatro valores, derivam-se métricas que capturam aspectos distintos do desempenho do modelo. A precisão mede a proporção de predições positivas que de fato correspondem a ataques, penalizando o modelo por falsos positivos:

$$\text{Precisão} = \frac{TP}{TP + FP}$$

A revocação, por sua vez, mede a proporção de ataques reais que o modelo consegue detectar, penalizando falsos negativos:

$$\text{Revocação} = \frac{TP}{TP + FN}$$

Em sistemas de detecção de intrusão, essas duas métricas frequentemente entram em conflito: um modelo mais permissivo tende a aumentar a revocação às custas da precisão, e vice-versa.

A medida F1 combina precisão e revocação em um único valor, por meio de sua média harmônica, atribuindo peso igual às duas:

$$F_1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}}$$

Quando um dos dois tipos de erro é considerado mais grave que o outro, é possível generalizar essa combinação por meio da medida F_β , que pondera a revocação β vezes mais que a precisão:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{(\beta^2 \cdot \text{Precisão}) + \text{Revocação}}$$

O caso particular $\beta = 2$, chamado de medida F2, atribui à revocação um peso consideravelmente maior que à precisão, refletindo cenários em que deixar de detectar um ataque é considerado mais custoso do que gerar um alerta incorreto.

2.6 Ferramentas do Ambiente de Teste

Esta seção apresenta as ferramentas de infraestrutura empregadas no ambiente containerizado descrito na Seção 4.3, cujo funcionamento é pressuposto ao longo da descrição desse ambiente.

2.6.1 CICFlowMeter

O CICFlowMeter é uma ferramenta de código aberto para geração e análise de fluxos de tráfego de rede, mantida pelo Canadian Institute for Cybersecurity. A partir de um arquivo de captura de pacotes, a ferramenta produz fluxos bidirecionais: o primeiro pacote observado entre dois hosts determina o sentido de ida (*forward*) da comunicação, e os pacotes subsequentes trocados no sentido inverso são agrupados no mesmo registro, compondo o sentido de volta (*backward*) desse fluxo (23).

Um fluxo é delimitado pela combinação de cinco atributos do cabeçalho dos pacotes: os endereços IP de origem e destino, as portas de origem e destino, e o protocolo de transporte utilizado. Todo pacote que compartilhe esses cinco valores com um fluxo já em andamento é agregado a esse mesmo registro, em vez de originar um novo. Um fluxo é encerrado quando a conexão é explicitamente finalizada ou quando um intervalo de inatividade pré-configurado é atingido sem que novos pacotes sejam observados (23, 24).

2.6.2 RabbitMQ e o Protocolo AMQP

O RabbitMQ é um *message broker*: um serviço intermediário que recebe mensagens de aplicações produtoras, encaminha-as de acordo com regras de roteamento definidas e as disponibiliza para consumo por aplicações consumidoras, sem que produtor e consumidor precisem se comunicar diretamente (25). Essa mediação permite que os dois lados operem de forma desacoplada, cada um em seu próprio ritmo, sem que um precise aguardar a disponibilidade do outro.

O RabbitMQ implementa o AMQP (*Advanced Message Queuing Protocol*), um protocolo de camada de aplicação que define como mensagens são publicadas, roteadas e entregues. No modelo do AMQP, uma aplicação produtora não publica uma mensagem diretamente em uma fila, mas sim em uma *exchange*, um componente responsável por rotear cópias da mensagem para uma ou mais filas, de acordo com regras de vinculação estabelecidas entre a *exchange* e essas filas. Uma vez na fila, a mensagem permanece disponível até ser entregue a um consumidor inscrito ou retirada por demanda (25).

2.6.3 Observabilidade: Métricas e Logs

O acompanhamento do funcionamento de um sistema em operação costuma se apoiar em duas fontes de dados complementares: métricas, que resumem numericamente o comportamento do sistema ao longo do tempo, e logs, que registram eventos discretos com maior riqueza de contexto. Cada uma dessas fontes é tipicamente coletada e consultada por meio de ferramentas próprias, especializadas nas características de cada tipo de dado.

O Prometheus é um sistema de monitoramento que coleta métricas na forma de séries temporais, isto é, sequências de valores numéricos associados a um instante de tempo. Sua coleta segue, por padrão, um modelo de *pull*: o próprio Prometheus consulta periodicamente um endpoint exposto por cada serviço monitorado, obtendo os valores mais recentes de suas métricas (26). Esse modelo, no entanto, não é adequado a processos de curta duração, que podem terminar antes de serem consultados. Para esses casos, o PushGateway atua como um intermediário: o próprio processo publica suas métricas nele antes de encerrar, e o Prometheus passa a consultar o PushGateway em vez do processo original (27).

Já o registro de eventos é tratado pelo Elastic Stack, um conjunto de ferramentas de código aberto voltado à ingestão, ao armazenamento e à visualização de dados em larga escala, composto centralmente por três componentes: o Logstash, responsável por coletar dados de múltiplas origens e transformá-los em um formato padronizado; o Elasticsearch, um mecanismo de busca e análise que armazena e indexa esses dados, permitindo consultas eficientes sobre grandes volumes; e o Kibana, que fornece uma interface para exploração e visualização dos dados armazenados no Elasticsearch (28).

3 TRABALHOS ANTERIORES

3.1 Impactos do pré-processamento de dados e da otimização de hiperparâmetros no desempenho de modelos de aprendizado de máquina aplicados a sistemas de detecção de intrusão

Lima et al. (6) investigaram o impacto das ações de pré-processamento de dados e da otimização de hiperparâmetros sobre o desempenho de modelos de aprendizado de máquina aplicados a Sistemas de Detecção de Intrusões. Os autores observaram que a literatura da área tendia a relatar essas etapas sem aprofundar sua influência efetiva nos resultados, e propuseram um estudo experimental para preencher essa lacuna.

Para conduzir a investigação, foram comparados três cenários experimentais: sem pré-processamento e sem otimização, com pré-processamento e sem otimização, e com ambos. Essas configurações foram aplicadas aos conjuntos de dados públicos CSE-CIC-IDS2018 e KDD Cup 1999. Cinco algoritmos foram avaliados, incluindo *Random Forest*, árvore de decisão, XGBoost, *Naive Bayes* e uma rede neural, com métricas de classificação e tempos de treinamento e teste registrados em cada combinação. Os resultados indicaram que a aplicação adequada das técnicas de pré-processamento, em conjunto com a otimização dos hiperparâmetros, produziu modelos mais robustos e reduziu de forma expressiva os tempos de execução, com destaque para os algoritmos *Random Forest*, árvore de decisão e XGBoost, que apresentaram desempenho preditivo consistentemente elevado.

A conclusão do estudo consolida uma diretriz prática para o desenvolvimento de Sistemas de Detecção de Intrusões baseados em aprendizado de máquina: investir em um *pipeline* de pré-processamento bem estruturado antes de partir para o ajuste fino de hiperparâmetros, sequência que produz modelos simultaneamente mais precisos e mais eficientes em termos de tempo de execução. Como contribuição complementar, os autores disponibilizaram um repositório público contendo os *scripts* de automação de pré-processamento, treinamento e teste, juntamente com as métricas e tempos de execução completos, viabilizando a reprodução dos experimentos e a continuidade da pesquisa em direção à aplicação dessas técnicas em redes operacionais reais.

3.2 Implementação de uma camada de proteção baseada em anomalias no Sistema de Detecção de Intrusão do Módulo de Proteção Cibernética utilizando técnicas de Aprendizado de Máquina

3.2.1 Projeto Final de Curso 2024

Santos e Cuadros (7) avançaram na direção da aplicação prática de técnicas de aprendizado de máquina à detecção de intrusões, tomando como ponto de partida a base metodológica estabelecida no trabalho anterior. O foco do projeto foi desenvolver uma abordagem automatizada para a análise de tráfego anômalo em redes operacionais, contemplando desde a seleção de algoritmos até a entrega de modelos prontos para uso.

O trabalho foi conduzido sobre o conjunto de dados CSE-CIC-IDS2018, adotado por ser de domínio público e amplamente utilizado, permitindo comparação de resultados. O projeto foi estruturado em torno de uma pipeline metodológica composta por quatro grandes etapas: (i) escolha de uma ferramenta de *Automated Machine Learning* (AutoML), responsável por apoiar a seleção das técnicas de aprendizado mais promissoras; (ii) pré-processamento dos dados, contemplando ações como limpeza, normalização e balanceamento por tipo de ataque; (iii) seleção dos algoritmos a serem treinados, orientada pelos resultados oferecidos pela ferramenta de AutoML; e (iv) refinamento dos hiperparâmetros desses algoritmos, com vistas à maximização do desempenho dos modelos resultantes.

Como resultado, os autores entregaram um conjunto de dados pré-processado e um conjunto de modelos treinados, com a finalidade de apoiar o desenvolvimento de uma nova abordagem de detecção de anomalias destinada ao MPC. Os códigos da pipeline de pré-processamento desenvolvida ao longo do projeto também foram disponibilizados, podendo ser reaproveitados em estudos posteriores. Além disso, os autores identificaram limitações na classificação de certas categorias de ataque, em particular Infiltração, Força Bruta Web e Injeção SQL, e apontaram a Mistura de Especialistas (Mixture of Experts, MoE) como direção promissora a ser explorada em trabalhos posteriores, em razão de sua capacidade de combinar modelos especializados em diferentes padrões de ataque.

3.2.2 Projeto Final de Curso 2025

Asfora e Soares (8) avançaram a partir das contribuições anteriores ao traduzir a estratégia de detecção baseada em anomalias em uma implementação operacional dentro do ambiente do MPC. O trabalho dedicou-se à preparação de um ambiente experimental representativo e à adaptação dos componentes de inferência previamente desenvolvidos, viabilizando a operação do modelo Mistura de Especialistas em paralelo aos demais

elementos do MPC.

Em termos de infraestrutura, os autores construíram uma versão virtualizada do MPC 1.0 sobre o Proxmox VE, com instâncias dedicadas para o pfSense e para o Security Onion. Sobre esse ambiente, foi implantada uma API denominada Oráculo como serviço containerizado independente, operando em paralelo ao Security Onion: ambos conectados à mesma ponte virtual, recebem o tráfego espelhado pelo pfSense, mas mantêm fluxos de processamento distintos. O Oráculo, denominação dada à API que reúne a pipeline de pré-processamento desenvolvida em trabalhos anteriores e o modelo Mistura de Especialistas previamente treinado, é responsável por receber, preparar e submeter o tráfego espelhado à inferência. O modelo, composto por um roteador e múltiplos especialistas, é acompanhado de uma etapa de calibração probabilística aplicada às suas saídas.

Como contribuição experimental, os autores conduziram a primeira comparação direta entre o Oráculo e o Suricata em ambiente controlado, utilizando capturas de tráfego com perfis benignos e de ataques de força bruta e negação de serviço sobre HTTP. O Oráculo apresentou alta sensibilidade na identificação do tráfego malicioso, ainda que com ruído residual em tráfego benigno, enquanto o Suricata não emitiu alertas sobre as amostras avaliadas, comportamento que os autores atribuem a possível inadequação das regras configuradas para os vetores em questão.

Em paralelo à comparação experimental, a etapa de automação do provisionamento do sistema virtualizado não se concluiu integralmente: a instalação desassistida do pfSense não pôde ser finalizada em razão de incompatibilidades na reconstrução da ISO de instalação, o que também postergou a configuração automatizada do Security Onion. Em síntese, o trabalho elevou a maturidade tecnológica da solução, deixando como pontos em aberto o refinamento da calibração do modelo, para mitigar os falsos positivos observados, e o aprimoramento do processo de provisionamento do ambiente virtualizado.

4.2 Virtualização do MPC 1.0

A primeira etapa metodológica consistiu na virtualização dos componentes do *Módulo de Proteção Cibernética* (MPC 1.0), de modo a replicar as condições de um ambiente embarcado isolado. Esse arranjo foi hospedado em um servidor dedicado físico da Hetzner, de uso exclusivo, em Helsinki, Finlândia. Toda a infraestrutura virtualizada (Proxmox + VMs) rodou sobre esse hardware real.

4.2.1 Automação do provisionamento

Com o objetivo de tornar o provisionamento do ambiente reproduzível e reduzir a intervenção manual necessária para sua implantação, foi desenvolvido um conjunto de *scripts* de automação estruturado em quatro componentes complementares. Os dois primeiros operam no host Proxmox e cobrem desde a verificação de pré-requisitos e a obtenção das imagens de instalação até a criação das *bridges* de rede e das máquinas virtuais, com suas respectivas configurações de hardware e conectividade.

Após a instalação base das VMs, os dois componentes restantes completam a configuração do ambiente. O terceiro configura o roteamento, o NAT e o espelhamento de tráfego garantindo que todas as definições sejam persistidas entre reinicializações. O quarto aplica as correções necessárias ao funcionamento do sistema de detecção dentro da VM do Security Onion.

Apesar do nível de automação alcançado, a instalação base do pfSense e do Security Onion e a configuração inicial deste último por meio de seu assistente ainda exigem interação manual, pois dependem de acesso ao console das VMs e de decisões que variam conforme o ambiente de destino. O conjunto de *scripts* encerra seu fluxo com a apresentação das instruções necessárias para que o operador conclua essas etapas e coloque o ambiente em operação. A Figura 3 descreve o organograma com o fluxo a ser realizado na implantação do ambiente virtualizado do MPC 1.0.

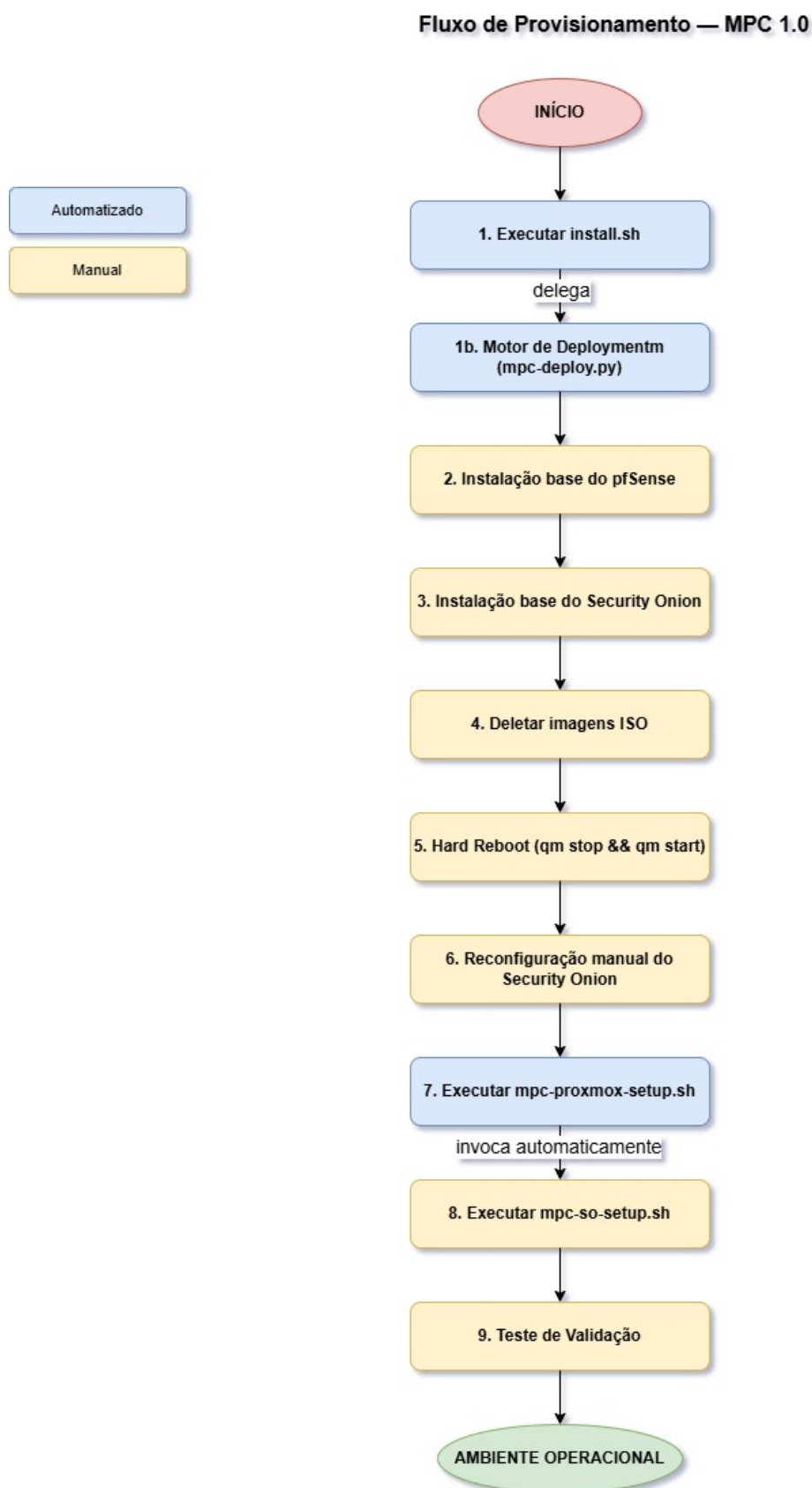


Figura 3 – Fluxograma que descreve o provisionamento do ambiente virtual do MPC 1.0

4.2.2 Fluxo de dados

A Figura 2 apresenta a topologia detalhada da infraestrutura virtualizada do MPC, com seus principais componentes e suas respectivas interfaces de rede. Em razão da ausência de uma segunda máquina virtual no cenário de testes, não foi possível verificar o tráfego saindo da rede externa e chegando a uma rede interna. Foi possível, contudo, comprovar o correto espelhamento do tráfego para o *Security Onion*, validando a função de captura passiva da arquitetura.

O roteamento é integralmente centralizado no *firewall* virtual pfSense (VM 100), que dispõe de quatro interfaces virtuais principais, cada uma mapeada em uma *Linux Bridge* distinta. A seguir, são descritas as principais características de cada interface.

- **BACKBONE1**, ligada ao **vmbr6003**, recebe tráfego interno e pode ser utilizada para fazer o controle de acesso de um segmento de rede, permitindo bloqueio ou liberação de mensagens, além de auditoria por meio do Security Onion.
- **WAN**, ligada ao **vmbr6001**, entrega o tráfego à rede externa.
- **MNGMT**, ligada ao **vmbr1**, utilizada para gerenciamento do sistema operacional (Proxmox) e da Interface Web do PFSense.
- **SECONIONGW**, ligada ao **vmbr5001**, usada para fornecer uma rota para outras redes poderem acessar a rede do Security Onion (rede 172.16.161.0/24). Além disso, é a rota padrão da máquina virtual IDS

O percurso lógico do pacote neste trabalho consiste em um pacote originado na rede externa, associada ao bridge **vmbr6001**. Esse bridge encaminha o quadro diretamente à interface *WAN* do pfSense. Em paralelo, o pfSense replica todos os pacotes que atravessam o **BACKBONE1**/*WAN* para a interface *SPAN*. Esse tráfego chega a bridge **vmbr5002** sendo consumido pela interface de *Sniffing* do Security Onion (VM 101) para serem analisados, permitindo inspeção profunda sem interferir no fluxo principal.

4.2.3 Especificações das máquinas virtuais

As máquinas virtuais que compõem o arranjo foram provisionadas com configurações específicas para representar a arquitetura do MPC 1.0. A Tabela 1 resume os recursos alocados (vCPU, memória, armazenamento e *bridges* de rede) para cada VM.

Os recursos foram alocados para equilibrar o desempenho de cada função (firewall, IDS) com as limitações típicas de ambientes embarcados e de laboratório.

Tabela 1 – Configurações das VMs provisionadas no *Proxmox*.

Máquina (ID)	vCPU	RAM (GiB)	Disco (GiB)
Proxmox-VE (host)	12	62.72	527.73
mpc-alpha-pfsense (<i>ID</i>)	2	2.00	20.00
mpc-alpha-securityonion (<i>ID</i>)	4	24.00	200.00

4.2.3.1 Papéis dos nós do ambiente

Cada instância do ambiente virtualizado foi concebida com um papel específico, conforme descrito a seguir:

- **Proxmox-VE:** núcleo de gerenciamento do módulo MPC, responsável por orquestrar políticas de segurança e interfaces administrativas.
- **Rede Interna:** simula clientes internos, gerando tráfego de teste na interface **eno3** para validar regras de firewall e NAT. Por meio da interface **eno2**, atua como rede de gerenciamento para acesso às interfaces de usuário da web para Promox (172.16.160.60:8006), pfSense (172.16.160.1) e Security Onion (172.16.161.60).
- **Rede externa - PC atacante:** atua como nó externo, enviando tráfego HTTP e payloads de ataque diretamente ao endereço público do servidor Proxmox. O tráfego é recebido pelo Proxmox, que através da bridge **vmbr6001**, encaminha os pacotes à interface WAN do pfSense (**vtnet0**).
- **VM 100 - pfSense:** perímetro de fronteira, implementa todas as políticas de roteamento, NAT e filtragem entre o segmento interno e externo.
- **VM 101 - Security Onion:** sensor IDS em modo passivo, recebe cópias de pacotes via SPAN para análise.

4.2.4 Configurações de interfaces de rede

Nesta seção, apresentam-se as configurações de rede das máquinas virtuais (VMs) que compõem o ambiente virtualizado. Cada VM foi configurada com múltiplas interfaces de rede, conectadas a *bridges* distintas, permitindo a separação lógica entre redes de gerenciamento, *backbone* e tráfego monitorado.

Tabela 2 – Interfaces da VM 100 (pfSense CE 2.7.2).

NIC (guest)	Bridge (host)	Papel	CIDR	Firewall
vtnet0	vmbr6001	WAN	10.0.0.2/30	1
vtnet1	mpc-alpha-mngmt	MNGMT	172.16.160.1/24	1
vtnet2	vmbr5001	SECONIONGW	172.16.161.1/24	1
vtnet5	mpc-alpha-mngmt	NA_MNGMT	DHCP	1
vtnet6	vmbr5002	SPAN	—	1

Tabela 3 – Interfaces da VM 101 (Security Onion 2.3.300).

NIC (guest)	Bridge (host)	Papel	CIDR	Firewall
eth0	vmbr5001	Gerenciamento	172.16.161.60/24	1
eth1	vmbr5002	Sniffer (SPAN)	—	0

4.2.5 Validação do ambiente

Para assegurar que o cenário virtualizado atenda aos requisitos de conectividade e detecção, foram executadas as seguintes verificações:

1. Verificação de conectividade básica entre Proxmox e pfSense:

- No shell do Proxmox, executou-se `ping 172.16.160.1` (Proxmox → pfSense MNGMT) e `ping 172.16.161.60` (Proxmox → Security Onion via pfSense), confirmando que o roteamento entre os segmentos internos está funcional.
- No shell do pfSense, executou-se `ping 172.16.161.60` (pfSense → Security Onion), confirmando conectividade direta na interface SECONIONGW.

2. Verificação do espelhamento de tráfego via tcpdump:

- Na interface `eth1` da Security Onion, executou-se `tcpdump -i eth1 -n port 80`, enquanto o PC atacante enviava requisições HTTP ao IP público do Proxmox na porta 8080.
- Confirmou-se a presença dos pacotes com origem `179.174.x.x` e destino `10.0.0.2:80`, demonstrando que o espelhamento encaminha corretamente cópias dos pacotes à interface de monitoramento da Security Onion.

3. Geração de tráfego malicioso e verificação de alertas no Suricata:

- O PC atacante enviou requisições HTTP com payloads conhecidos, nomeadamente uma tentativa de *Shellshock* e uma sondagem de arquivo `.env`:

```
curl "http://<IP_PUBLICO>:8080/cgi-bin/test.cgi" \
-H "User-Agent: () { :; }; /bin/bash -i >& /dev/tcp/1.2.3.4/4444
↪ 0>&1"

curl "http://<IP_PUBLICO>:8080/.env"
```

- No arquivo `eve.json` do contentor `so-suricata`, verificou-se a geração automática de dois alertas:
 - ET WEB_SERVER Possible bash shell piped to dev tcp Inbound to WebServer (severidade: média)
 - ET INFO Request to Hidden Environment File - Inbound (severidade: baixa)
- Os mesmos alertas foram posteriormente confirmados na interface web da Security Onion em **Alerts** → **Today**, validando o pipeline completo de detecção.

4.3 Ambiente de Teste

Em paralelo ao ambiente virtualizado do MPC 1.0 descrito na Seção 4.2, esta etapa do trabalho parte do ambiente containerizado desenvolvido por Asfora e Soares (8), dedicado à avaliação do Oráculo e do modelo Mixture of Experts nele integrado, assim como ao fluxo de dados percorrido pelo Suricata. Diferentemente do ambiente da Seção 4.2, baseado em máquinas virtuais sobre o Proxmox VE, esse ambiente é implementado por meio de contêineres Docker, organizados segundo suas responsabilidades funcionais e orquestrados via `docker-compose`.

4.3.1 Visão Geral do Pipeline

O pipeline é organizado em quatro camadas funcionais. A camada de ingestão é responsável por alimentar o sistema com tráfego de rede a partir de arquivos `.pcap` e por extrair, desse tráfego, os atributos que servirão de entrada ao modelo de classificação. A camada de classificação corresponde ao próprio Oráculo, responsável por processar os atributos extraídos e emitir alertas quando identifica tráfego malicioso. Em paralelo, uma camada de detecção por assinaturas executa o Suricata sobre o mesmo tráfego, permitindo comparar os alertas gerados pela abordagem baseada em assinaturas com os gerados pela abordagem baseada em aprendizado de máquina. Por fim, uma camada de observabilidade registra métricas de operação e eventos estruturados ao longo de todo o pipeline, apoiando tanto o acompanhamento do funcionamento do sistema quanto a investigação de resultados. A Figura 4 ilustra essas quatro camadas e o fluxo de dados entre elas. As seções seguintes detalham cada uma delas.

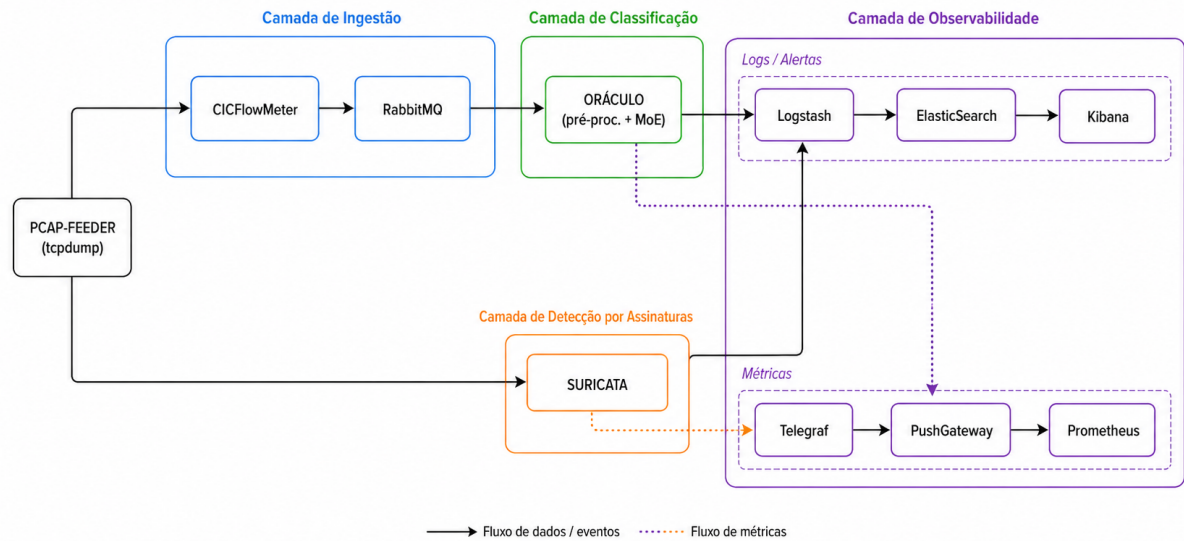


Figura 4 – Camadas funcionais do ambiente de testes.

4.3.2 Ingestão e Pré-processamento

A camada de ingestão é composta por três componentes: o **pcap-feeder**, o **CIC-FlowMeter** e o **RabbitMQ**, responsáveis, respectivamente, por disponibilizar o tráfego de rede ao pipeline, extrair dele os atributos relevantes para classificação e disponibilizar essas amostras de forma assíncrona ao componente de classificação.

O **pcap-feeder** é o ponto de entrada do pipeline, funcionando como um distribuidor de tráfego, encaminhando os pacotes para os componentes responsáveis por interpretá-los. Seu comportamento é determinado por variáveis de ambiente, das quais a mais relevante é **CAPTURE**:

- **CAPTURE=0** (modo *replay*): o contêiner monitora um diretório de entrada em busca de arquivos **.pcap** já existentes e os copia, sem qualquer alteração de conteúdo, para os diretórios monitorados pelos componentes seguintes;
- **CAPTURE=1** (modo *live*): uma instância do **tcpdump** é ativada sobre uma interface de rede especificada, capturando tráfego continuamente e gerando, a partir dele, novos arquivos **.pcap**.

Em ambos os modos, a saída consiste em arquivos **.pcap** depositados em um ou mais diretórios de destino, conforme variáveis de ambiente que habilitam o espelhamento simultâneo do tráfego para a camada de classificação e para a camada de detecção por assinaturas.

No ambiente estudado, o CICFlowMeter concentra duas etapas do pipeline: a extração de atributos de fluxo e a publicação dessas amostras no sistema de mensageria. Um script de monitoramento, `watchdog.sh`, observa continuamente o diretório de entrada, onde os arquivos `.pcap` são depositados pelo `pcap-feeder`, e aciona automaticamente a execução do CICFlowMeter assim que um novo arquivo é detectado.

Essa ferramenta converte o arquivo `.pcap` recebido em um arquivo `.csv`, agrupando os pacotes em fluxos bidirecionais. Cada fluxo reúne, em um único registro, os pacotes trocados nos dois sentidos de uma mesma conexão, calculando estatísticas separadas para o sentido de ida e de volta, como quantidade de pacotes, tamanho médio e tempos entre chegadas. Esses atributos são organizados em duas categorias: identificadores, como IPs, portas, protocolo e instante de captura, e atributos estatísticos propriamente ditos.

Após a geração do `.csv`, um script de serialização, `producer.py`, realiza sua leitura linha por linha do arquivo, usando o cabeçalho para identificar automaticamente cada campo. Os metadados do fluxo, como IPs e portas, são mantidos separados do vetor de atributos numéricos, e o conjunto é encapsulado em um objeto JSON. Esse objeto é então publicado, via protocolo AMQP, em uma fila dedicada do RabbitMQ.

O RabbitMQ atua como o sistema de mensageria do pipeline, recebendo as amostras publicadas pelo CICFlowMeter em uma fila dedicada, denominada `model-queue`, e mantendo-as disponíveis, no mesmo formato JSON em que foram publicadas, até que sejam consumidas pelo Oráculo. Essa fila funciona como um buffer assíncrono entre a geração das amostras e sua análise, desacoplando a velocidade de extração de atributos da velocidade de inferência do modelo, de modo que variações no volume de tráfego processado não comprometam a estabilidade do pipeline.

4.3.3 Camada de Classificação

O Oráculo constitui a camada de classificação do pipeline, responsável por consumir os fluxos de rede publicados na fila de mensageria e realizar a inferência sobre cada amostra com base no modelo Mixture of Experts. Sua arquitetura interna é modular, dividida em componentes que atuam em sequência: o `MessageBroker`, responsável pela conexão com o RabbitMQ e pelo consumo contínuo da fila; o `MessengerService`, responsável por orquestrar o processamento de cada mensagem recebida; o `ClassificationService`, responsável por extrair e normalizar os atributos da amostra; o `Predictor`, responsável pela inferência através do modelo de roteamento (*gate*) e dos especialistas do MoE e, por fim, o `PackageService`, responsável por construir o registro do alerta e delegá-lo a um repositório de persistência quando um tráfego malicioso é identificado.

A entrada de dados no Oráculo ocorre por meio da fila `model-queue` do RabbitMQ, alimentada externamente pelo CICFlowMeter a cada fluxo de rede processado. Cada

mensagem é entregue, em formato JSON, contendo o endereço IP de origem, a porta de origem, o endereço IP de destino, um identificador do fluxo e um vetor de atributos numéricos extraídos pelo CICFlowMeter. Ao ser recebida pelo **MessageBroker**, a mensagem é repassada, via função de *callback*, ao **MessengerService**, responsável por orquestrar as etapas subsequentes de processamento daquela amostra.

A mensagem recebida é encaminhada ao **ClassificationService**, responsável por convertê-la em uma entrada compatível com o modelo de classificação. A partir do JSON recebido, são extraídos o endereço IP de origem, o identificador do fluxo e o vetor de atributos numéricos, que é então normalizado. Esse vetor, acompanhado do identificador e da estratégia de *gating* configurada para o sistema, é repassado ao **Predictor** para a etapa de inferência.

A inferência é realizada pelo **Predictor**, que implementa a abordagem Mixture of Experts. O modelo de roteamento *gate*, previamente calibrado, estima, para cada classe reconhecida pelo sistema, a probabilidade de a amostra pertencer àquela categoria. Na estratégia de *gating* padrão do sistema, apenas os especialistas cuja probabilidade estimada pelo gate ultrapassa um limiar são executados sobre a amostra e os escores retornados por eles são então normalizados entre si. Por fim, a classe de maior escore normalizado é adotada como predição final, acompanhada de sua respectiva confiança.

A partir do resultado da classificação, isto é, a classe de maior escore e sua confiança associada, cabe ao **MessengerService** avaliar se a amostra representa tráfego malicioso. Caso a classe predita seja diferente de *Benign*, o **MessengerService** aciona o **PackageService**, repassando o endereço IP de origem, o identificador do fluxo, o tipo de ataque inferido e a confiança da predição. O **PackageService** constrói então o registro do alerta, associando a esses dados um *timestamp* referente ao instante da inferência, e delega sua persistência a um repositório. Quando a predição corresponde a tráfego benigno, nenhum alerta é gerado e a amostra é descartada.

4.3.4 Camada de Detecção por Assinaturas

O Suricata compõe a camada de detecção por assinaturas, executando sobre o mesmo tráfego processado pelo Oráculo de modo a permitir a comparação entre os alertas gerados pela abordagem baseada em assinaturas e os gerados pela abordagem baseada em aprendizado de máquina.

A entrada do Suricata consiste nos mesmos arquivos *.pcap* depositados pelo *pcap-feeder*, encaminhados a um diretório próprio, sem qualquer transformação prévia. O Suricata processa esses arquivos comparando o conteúdo dos pacotes contra um conjunto de regras de detecção, mantido atualizado a partir de duas fontes públicas: o conjunto *Emerging Threats Open* e o *Snort Community Rules*.

Ao identificar uma correspondência entre o tráfego analisado e uma das regras carregadas, o Suricata registra um evento de alerta em um arquivo `eve.json`, no formato NDJSON, em que cada linha corresponde a um evento independente, contendo, entre outros campos, os endereços IP e portas de origem e destino, o protocolo, e a assinatura da regra correspondente ao alerta.

Esse arquivo é consumido por dois caminhos paralelos: pelo Logstash, que o lê continuamente e encaminha cada evento para indexação no Elasticsearch, e pelo Telegraf, que extrai métricas operacionais do Suricata, como a contagem de *flows* e de alertas disparados, e as expõe em um endpoint próprio, consultado periodicamente pelo Prometheus.

4.3.5 Camada de Observabilidade

A camada de observabilidade recebe dados tanto do Oráculo quanto do Suricata, organizando-os em dois caminhos paralelos. Um deles é dedicado a logs e alertas, apoiado no Elastic Stack, enquanto o outro é dedicado a métricas, apoiado no Prometheus.

No caminho de logs, o repositório de persistência do Oráculo publica cada registro de alerta via TCP, em formato JSON, para uma configuração do Logstash dedicada a essa origem. Essa configuração reconhece as mensagens do Oráculo pelo campo `ids` e as transforma antes da indexação. O endereço IP é copiado e renomeado para o campo aninhado `source.ip`, o identificador do fluxo é renomeado para `pcap.id`, o tipo de ataque inferido é renomeado para `classification`, e o *timestamp* da inferência é convertido para o campo de tempo padrão do Elasticsearch. O documento resultante é indexado em um índice próprio, dedicado exclusivamente aos alertas do Oráculo.

Já os eventos do Suricata chegam ao Logstash de um jeito diferente, pois, em vez de serem publicados ativamente como no Oráculo, são lidos diretamente do arquivo `eve.json` por uma configuração do Logstash que monitora esse arquivo continuamente. Essa configuração não transforma os campos do evento, de modo que cada alerta é indexado praticamente no mesmo formato em que o Suricata o gera originalmente.

Como resultado, os alertas do Oráculo e os eventos do Suricata acabam armazenados separadamente no Elasticsearch, cada um com sua própria estrutura de campos. Isso significa que, atualmente, não há como cruzar automaticamente um alerta do Oráculo com um evento do Suricata referente ao mesmo tráfego e, assim, qualquer comparação entre os dois exigiria um trabalho manual de correspondência entre os registros.

No caminho de métricas, o Oráculo publica ativamente uma série de valores no PushGateway a cada predição realizada pelo **Predictor**, independentemente de a amostra ser classificada como maliciosa ou benigna. São enviadas, como valores numéricos, a latência da predição, o limiar de ativação dos especialistas e o tamanho do lote processado.

A estratégia de *gating* empregada acompanha essas métricas como um rótulo, permitindo segmentar os dados por estratégia. O Suricata, por sua vez, não publica métricas ativamente, pois um agente Telegraf lê periodicamente os eventos de estatísticas do arquivo `eve.json`, extraindo métricas como a contagem de fluxos totais e ativos, o uso de memória e a quantidade de alertas disparados e de regras carregadas, expondo-as em um endpoint próprio. O Prometheus consolida as duas origens consultando periodicamente tanto o PushGateway quanto o endpoint exposto pelo Telegraf.

4.4 Investigação do Pipeline

Esta seção apresenta uma investigação conduzida sobre os resultados experimentais obtidos por Asfora e Soares (8) na comparação entre o Oráculo e o Suricata. Ao revisar a metodologia empregada pelos autores, identificou-se que o total de fluxos utilizado como referência para o cálculo das métricas de ambas as ferramentas foi obtido a partir de uma única fonte, sem levar em conta que CICFlowMeter e Suricata contabilizam fluxos de rede de forma distinta sobre o mesmo tráfego. Essa constatação motivou uma análise mais aprofundada sobre como cada ferramenta define e delimita um fluxo, e sobre em que medida essa diferença compromete a coerência metodológica da comparação original. A partir desse estudo, avalia-se ainda, com base no escopo do experimento original, uma explicação para o desempenho observado do Suricata, testada empiricamente por meio da repetição do experimento com um novo conjunto de regras.

4.4.1 Definição de Fluxo: CICFlowMeter vs. Suricata

Embora tanto o CICFlowMeter quanto o Suricata rastreiem fluxos de rede a partir de um critério de identificação equivalente, a 5-tupla composta pelos endereços IP de origem e destino, as portas de origem e destino e o protocolo de transporte (29, 24), as duas ferramentas adotam critérios estruturalmente diferentes para determinar quando um fluxo é encerrado.

No código-fonte do CICFlowMeter, o encerramento por inatividade é calculado a partir do instante de início do fluxo, isto é, um fluxo é finalizado quando o tempo decorrido desde seu primeiro pacote ultrapassa um limiar fixo, independentemente de haver atividade contínua nesse intervalo (23). Já a documentação oficial do Suricata especifica que esse mesmo critério é calculado a partir do instante da última atividade observada no fluxo, e não do início da conexão, variando ainda conforme o estado da conexão, sendo mais curto para conexões recém-iniciadas e mais longo para conexões já estabelecidas (29).

Essa diferença tem uma consequência prática. Um fluxo longo e continuamente ativo, mesmo que configurado com o mesmo valor numérico de *timeout* nas duas ferramentas, seria tratado de forma distinta por cada uma delas. No CICFlowMeter, esse fluxo seria

fragmentado em múltiplos registros, simplesmente por exceder a duração total permitida desde seu início. Por outro lado, no Suricata, o mesmo fluxo permaneceria como um único registro contínuo, desde que a atividade não cessasse. Por isso, o número de fluxos identificados por cada ferramenta sobre um mesmo tráfego de rede não é diretamente comparável, ainda que ambas partam do mesmo critério de identificação de conexão.

4.4.2 Contagem de Fluxos e Alertas

A diferença estrutural apresentada na seção anterior tem uma implicação metodológica direta sobre como o desempenho de cada ferramenta deve ser avaliado. Como CICFlowMeter e Suricata não produzem, sobre o mesmo tráfego, a mesma contagem total de fluxos, o cálculo de métricas como a taxa de falsos negativos ou de verdadeiros negativos exige que cada ferramenta seja avaliada em relação ao total de fluxos que ela mesma identificou.

Uma segunda questão de granularidade também emerge dessa mesma investigação, relacionada à contagem de verdadeiros positivos. O Suricata registra cada correspondência entre o tráfego analisado e uma regra de detecção como um evento independente no arquivo `eve.json`. Como um mesmo fluxo pode corresponder a mais de uma regra, ou disparar a mesma regra mais de uma vez ao longo de sua duração, um único fluxo malicioso pode gerar diversas linhas de alerta. Isso significa que contar eventos de alerta, em vez de contar fluxos distintos que geraram ao menos um alerta, pode não refletir com precisão a quantidade real de fluxos maliciosos detectados.

4.4.3 O Dataset CICIoMT2024

O CICIoMT2024 é um conjunto de dados desenvolvido pelo Canadian Institute for Cybersecurity com o propósito de apoiar a avaliação de soluções de segurança voltadas à Internet das Coisas Médicas (Internet of Medical Things, IoMT), categoria de dispositivos conectados empregada no monitoramento e na assistência à saúde (30). O conjunto foi construído a partir de um ambiente de testes composto por 40 dispositivos IoMT, sendo 25 dispositivos reais e 15 simulados, comunicando-se por meio de protocolos amplamente utilizados nesse tipo de dispositivo, como Wi-Fi, MQTT e Bluetooth.

Sobre esse ambiente, foram executados 18 ataques distintos, agrupados em cinco categorias: negação de serviço distribuída (DDoS), negação de serviço (DoS), reconhecimento (*reconnaissance*), ataques específicos ao protocolo MQTT e falsificação de identidade (*spoofing*). Além do tráfego de ataque, o conjunto também contempla o comportamento normal dos dispositivos ao longo de seu ciclo de vida na rede, desde a entrada até a saída, permitindo distinguir anomalias de comportamento legítimo.

Os três arquivos utilizados no experimento conduzido por Asfora e Soares (8), um

de tráfego benigno, um de varredura de portas e um de negação de serviço por inundação SYN, pertencem, respectivamente, ao tráfego benigno geral do conjunto e às categorias de reconhecimento e de negação de serviço.

4.4.4 Conjunto de Regras Utilizado

O experimento conduzido pelos autores configurou o Suricata com o conjunto de regras Emerging Threats Open, complementado pelo Snort Community Rules.

O Emerging Threats Open é um conjunto de regras de detecção gratuito, distribuído sob licença BSD, compatível tanto com o Suricata quanto com o Snort, e atualizado com frequência (31). Trata-se de um dos conjuntos de regras mais amplamente utilizados na comunidade de código aberto, sendo inclusive recomendado pela própria documentação oficial do Suricata como referência de exemplos de assinaturas (29). Seu foco declarado é primariamente a detecção de *malware*, com regras derivadas da análise contínua de amostras coletadas e de indicadores de ameaça observados globalmente, refletindo majoritariamente cenários de ameaça típicos de redes de produção corporativas.

O Snort Community Rules é um subconjunto gratuito e de licença aberta do conjunto de regras do Snort, mantido e disponibilizado publicamente pela comunidade de usuários do Snort. Assim como o Emerging Threats Open, é amplamente utilizado como complemento a outros conjuntos de regras em implantações de código aberto. Seu foco também recai majoritariamente sobre ameaças típicas de redes corporativas, como comunicação de malware já instalado, exploração de vulnerabilidades em servidores de aplicação e banco de dados, e violações de políticas de uso de software.

Com base no escopo do CICIoMT2024 apresentado na Seção 4.4.3, uma inspeção das regras dos dois conjuntos relacionadas aos ataques de varredura de portas e de negação de serviço, presentes no experimento de Asfora e Soares (8), revela critérios de correspondência mais restritivos do que o necessário para identificar as variantes mais simples desses ataques. A regra voltada à detecção de inundação SYN (`sid:2019404`), por exemplo, exige, além das características do próprio pacote SYN, que o tamanho do *payload* exceda 900 bytes, uma condição que ferramentas de geração de tráfego de ataque mais simples, sem carga útil significativa, não satisfazem. De forma semelhante, a regra voltada à detecção de varredura de portas (`sid:2009582`) reconhece especificamente a assinatura de pacotes gerados pelo Nmap, ferramenta amplamente utilizada para varredura de portas e serviços em uma rede, configurado com uma janela TCP de valor 1024, restringindo a detecção a ferramentas de varredura que produzam pacotes com essa configuração exata.

Esse padrão sugere que a ausência de detecção observada no experimento não decorre de uma limitação da ferramenta Suricata em si, mas da adequação do conjunto de regras carregado ao tipo específico de tráfego analisado. Tanto o Emerging Threats Open

quanto o Snort Community Rules são moldados majoritariamente por ameaças observadas em redes de produção corporativas, enquanto o tráfego avaliado no experimento é sintético, gerado em ambiente controlado, com o propósito específico de servir de referência para testes de detecção. Essa diferença de origem ajuda a explicar por que regras desenvolvidas para reconhecer assinaturas de ferramentas ou condições de tráfego específicas, observadas nesse outro contexto, não correspondam necessariamente às características do tráfego presente no CICIoMT2024.

5 EXPERIMENTO

Este capítulo descreve os experimentos conduzidos para testar a hipótese levantada na investigação apresentada no Capítulo 4, segundo a qual o desempenho reduzido do Suricata, reportado por Asfora e Soares (8), decorre da inadequação do conjunto de regras de detecção utilizado ao tipo de tráfego presente no CICIoMT2024, e não de uma limitação da ferramenta em si. Validar essa hipótese é relevante para adequar o contexto da comparação entre o Suricata e o Oráculo, objetivo mais amplo deste trabalho.

Os experimentos são conduzidos em duas etapas. Na primeira, o experimento original é replicado nas mesmas condições relatadas pelos autores. Na segunda, mantêm-se o mesmo ambiente e os mesmos arquivos de tráfego, alterando-se o conjunto de regras carregado pelo Suricata.

5.1 Replicação do Experimento Original

Nesta etapa, o experimento conduzido por Asfora e Soares (8) é replicado utilizando o mesmo ambiente containerizado descrito no Capítulo 4, os mesmos três arquivos de tráfego (`Benign_test.pcap`, `Recon-Port_Scan_test.pcap` e `TCP_IP-DoS-SYN_test.pcap`) e o mesmo conjunto de regras originalmente carregado no Suricata, composto pelo Emerging Threats Open e pelo Snort Community Rules.

Essa etapa tem dois objetivos principais. Primeiro, confirmar que o ambiente utilizado neste trabalho reproduz os resultados relatados pelos autores antes de qualquer alteração ser introduzida, servindo como controle para as comparações realizadas na etapa seguinte. Segundo, coletar a quantidade de fluxos distintos que dispararam ao menos um alerta, em vez do número total de eventos de alerta registrados, conforme discutido na Seção 4.4.2. Essa contagem é obtida a partir dos eventos do tipo `flow` registrados pelo próprio Suricata no arquivo `eve.json`, que incluem um campo booleano indicando se algum alerta foi disparado ao longo daquele fluxo (29).

5.2 Experimento com Novo Conjunto de Regras

Nesta etapa, mantêm-se o mesmo ambiente containerizado e os mesmos três arquivos de tráfego utilizados na etapa anterior, alterando-se exclusivamente o conjunto de regras carregado pelo Suricata. Em vez do Emerging Threats Open e do Snort Community Rules, elaborou-se, com o auxílio do modelo Claude Sonnet 5, da Anthropic, um conjunto de regras próprio, voltado especificamente aos ataques de varredura de portas (*Port Scan*) e de negação de serviço por inundação SYN (*DoS SYN Flood*) presentes no experimento.

A elaboração desse conjunto partiu de uma análise estatística dos pacotes presentes nos arquivos de treino `Benign_train.pcap`, `Recon-Port_Scan_train.pcap` e `TCP_IP-DoS-SYN_train.pcap`. Essa análise identificou que o tráfego malicioso de ambos os ataques se concentra predominantemente em combinações específicas de *flags* TCP associadas a pacotes sem carga útil, característica pouco frequente no tráfego benigno correspondente. A taxa de pacotes por segundo originados de um mesmo endereço IP também se mostrou um discriminador robusto entre tráfego malicioso e benigno. Com base nesses padrões, as regras customizadas foram geradas e estão apresentadas no Código 5.1. As regras foram então aplicadas aos arquivos de teste correspondentes, garantindo que sua avaliação ocorresse sobre dados não consultados durante sua elaboração.

Código 5.1 – Conjunto de regras Suricata customizado.

```
alert tcp-pkt any any -> any any (msg:"CUSTOM SYN puro em alto volume";
  flags:S; dsize:0; detection_filter: track by_src, count 5, seconds 1;
  sid:9000001; rev:3;)

alert tcp-pkt any any -> any any (msg:"CUSTOM ACK+RST sem payload - possivel
  Port Scan";
  flags:AR; dsize:0; sid:9000002; rev:1;)

alert tcp-pkt any any -> any any (msg:"CUSTOM SYN-ACK em alto volume";
  flags:SA; dsize:0; detection_filter: track by_dst, count 5, seconds 1;
  sid:9000003; rev:3;)
```

6 RESULTADOS

Este capítulo apresenta os resultados obtidos nos dois experimentos descritos no Capítulo 5. A Seção 6.1 apresenta os resultados da replicação do experimento original, enquanto a Seção 6.2 apresenta os resultados obtidos com o novo conjunto de regras.

6.1 Resultados da Replicação

A Tabela 4 apresenta os resultados obtidos na replicação do experimento original, operando com o conjunto de regras Emerging Threats Open e Snort Community Rules, para cada um dos três arquivos de tráfego avaliados.

Tabela 4 – Resultados da replicação do experimento original.

Arquivo	Pacotes	Fluxos	Fluxos com alertas	Alertas
Benign_test.pcap	377.536	6.121	2	284
Recon-Port_Scan_test.pcap	226.233	211.922	13	75
TCP_IP-DoS-SYN_test.pcap	9.854.644	128.381	0	0
Total	10.458.413	346.424	15	359

O total de fluxos identificado nativamente pelo Suricata nos três arquivos, 346.424, é consideravelmente inferior ao total de 517.108 fluxos utilizado como referência tanto para o Suricata quanto para o Oráculo no experimento original de Asfora e Soares (8). Esse resultado confirma empiricamente a divergência estrutural discutida na Seção 4.4.1. As duas ferramentas, de fato, não contabilizam a mesma quantidade de fluxos sobre um mesmo tráfego, o que reforça a observação levantada na Seção 4.4.2 sobre a coerência entre resultado e fonte de contagem.

Os números também destacam uma segunda questão discutida naquela seção, relacionada à granularidade das detecções. No arquivo de tráfego benigno, por exemplo, apenas 2 fluxos distintos dispararam algum alerta, ainda que esses fluxos tenham gerado, ao todo, 284 eventos de alerta. Essa diferença ilustra bem a distinção entre contar eventos de alerta e contar o número de fluxos que dispararam ao menos um alerta, já que um mesmo fluxo pode disparar a mesma regra repetidas vezes ao longo de sua duração.

A partir da contagem de fluxos distintos que dispararam alerta, é possível recalcular as métricas de desempenho do Suricata no experimento original. A Tabela 5 apresenta esses valores.

Tabela 5 – Métricas do Suricata recalculadas a partir da contagem nativa de fluxos.

IDS	Fluxos	TP	FP	FN	TN	Precisão	Revocação	F1	F2
Suricata	346.424	13	2	340.290	6.119	0,8667	0,0000	0,0001	0,0000

6.2 Resultados com o Novo Conjunto de Regras

A Tabela 6 apresenta os resultados obtidos realizando o experimento com o conjunto de regras específicas, para cada um dos três arquivos de tráfego avaliados.

Tabela 6 – Resultados obtidos com o novo conjunto de regras.

Arquivo	Pacotes	Fluxos	Fluxos com alertas	Alertas
Benign_test.pcap	377.536	6.121	151	153
Recon-Port_Scan_test.pcap	226.233	211.903	211.382	211.421
TCP_IP-DoS-SYN_test.pcap	9.854.644	128.487	128.195	7.451.069
Total	10.458.413	346.511	339.728	7.662.643

A partir da contagem de fluxos distintos que dispararam alerta, é possível calcular as métricas de desempenho do Suricata. A Tabela 7 apresenta esses valores.

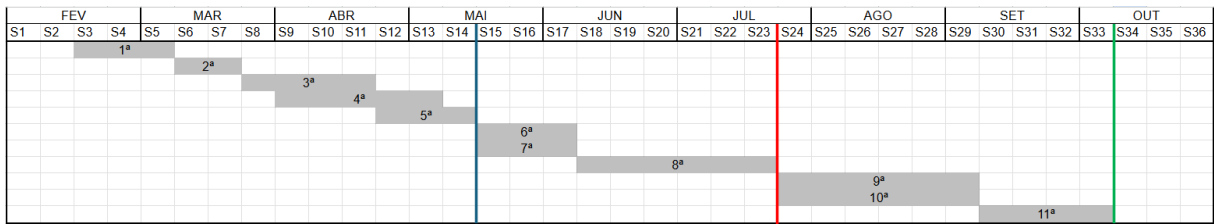
Tabela 7 – Métricas do Suricata com o novo conjunto de regras.

IDS	Fluxos	TP	FP	FN	TN	Precisão	Revocação	F1	F2
Suricata	346.511	339.577	151	813	5.970	0,9996	0,9976	0,9986	0,9980

As quatro métricas calculadas, Precisão de 0,9996, Revocação de 0,9976, F1 de 0,9986 e F2 de 0,9980, contrastam de forma acentuada com os valores obtidos na replicação do experimento original, na Seção 6.1. O conjunto de regras específicas, derivado da análise estatística dos arquivos de treino correspondentes a cada ataque, elevou a Revocação de valores próximos de zero para 0,9976, ao custo de um aumento nos falsos positivos sobre o tráfego benigno, de 2 para 151 fluxos. Esse resultado indica que a maior parte dos fluxos maliciosos deixados de lado pelo conjunto de regras original não escapava por uma limitação do motor de detecção do Suricata, mas por essas regras não reconhecerem o padrão específico de tráfego presente nesses ataques.

7 CRONOGRAMA

Esta seção apresenta a organização do cronograma do projeto, estruturado em etapas distribuídas ao longo do período previsto para seu desenvolvimento. A Figura 5 apresenta o gráfico de Gantt correspondente ao planejamento anual, no qual cada mês foi dividido em quatro unidades para representar a evolução das atividades ao longo do tempo.



11º: Teste e análise comparativa de desempenho entre o Oráculo e o Suricata

Datas Críticas:

1ª: Apresentação de **VE**: 15 de Maio

Serão apresentados:

- Ambiente do MPC 1.0 virtualizado.
- *Script* de automação para o provisionamento do ambiente do MPC 1.0 virtualizado e sua validação
- Manual do usuário a ser disponibilizado para as OMs de capacitação de pessoal

2ª: Apresentação de **VC**: semana do dia 14 de Agosto

Será apresentado o resultado da investigação da pipeline do Oráculo com o módulo MoE integrado, juntamente com propostas de ações para mitigar os resultados destoantes observados em pesquisas anteriores

3ª: Apresentação de **VF**: semana do dia 8 de outubro

Serão apresentados:

- Resultado dos testes realizados com fluxo de dados normalizado.
- Comparação entre Oráculo com MoE e Suricata, servindo como prova de conceito para o MPC 2.0

8 CONCLUSÃO

Este trabalho deu continuidade à evolução do Módulo de Proteção Cibernética, MPC 1.0, atuando em duas frentes complementares: a construção de um ambiente virtualizado representativo do MPC sem recorrer à virtualização aninhada, e o estudo da pipeline de detecção do Oráculo, módulo baseado na arquitetura Mixture of Experts, em comparação com o Suricata.

Na frente de infraestrutura, foi construído um cenário virtualizado composto por uma instância do pfSense, responsável pelo roteamento, pela filtragem e pelo espelhamento de tráfego, e por uma instância do Security Onion, responsável pelo monitoramento passivo e pela análise dos eventos de segurança. Sobre esse cenário, desenvolveu-se um conjunto de scripts que automatiza a maior parte do provisionamento, cobrindo desde a criação das máquinas virtuais e das bridges de rede até a configuração do roteamento, do NAT e do espelhamento de tráfego. A instalação base do pfSense e do Security Onion, assim como a configuração inicial deste último por meio de seu assistente, ainda exige interação manual no console das VMs, de modo que o conjunto de scripts encerra seu fluxo apresentando ao operador as instruções necessárias para concluir essas etapas. A validação do ambiente confirmou o funcionamento correto da conectividade entre os segmentos de rede, do espelhamento de tráfego para a interface de monitoramento e da geração de alertas pelo Suricata a partir de payloads maliciosos conhecidos.

Na frente de detecção, o trabalho partiu do ambiente containerizado herdado de Asfora e Soares (8), estudando em profundidade sua pipeline de ingestão, classificação e observabilidade, e voltou-se em seguida à análise dos resultados experimentais obtidos por esses autores na comparação entre o Oráculo e o Suricata, um dos pontos que havia permanecido em aberto ao final daquele trabalho. Essa análise revelou uma inconsistência metodológica na base dos resultados: o total de fluxos utilizado para calcular as métricas de ambas as ferramentas fora extraído de uma única fonte, sem considerar que o CICFlowMeter e o Suricata adotam critérios distintos para encerrar um fluxo, o primeiro contando o tempo de inatividade a partir do início da conexão e o segundo a partir de sua última atividade observada. Identificou-se ainda uma segunda fonte de imprecisão, relacionada à contagem de alertas: como o Suricata registra cada correspondência de regra como um evento independente no arquivo eve.json, contar esses eventos em vez de fluxos distintos que dispararam ao menos um alerta pode distorcer a leitura do desempenho da ferramenta.

A partir dessas observações, investigou-se uma explicação para o desempenho consideravelmente baixo do Suricata reportado no experimento original. A análise do conjunto de regras utilizado, composto pelo Emerging Threats Open e pelo Snort Community Rules, mostrou que essas regras são moldadas majoritariamente por ameaças típicas de redes

corporativas, e não pelo tráfego sintético e controlado do CICIoMT2024 empregado no experimento. Duas regras relacionadas aos ataques testados, a de inundação SYN e a de varredura de portas, ilustram esse descompasso, exigindo condições de tráfego, como um payload mínimo ou a assinatura de uma ferramenta específica, que não correspondem às características dos ataques presentes no dataset.

Essa hipótese foi testada empiricamente em duas etapas. Primeiro, replicou-se o experimento original, o que confirmou a divergência na contagem de fluxos entre as duas ferramentas: o Suricata identificou nativamente 346.424 fluxos nos três arquivos avaliados, valor consideravelmente inferior aos 517.108 fluxos utilizados como referência no experimento original. A partir dessa contagem nativa, recalcularam-se as métricas do Suricata, obtendo-se uma precisão de 0,8667 e uma revocação próxima de zero, o que confirmou o baixo desempenho relatado originalmente, agora calculado sobre uma base metodologicamente coerente. Em seguida, aplicou-se aos mesmos arquivos de teste um novo conjunto de regras, elaborado a partir da análise estatística dos arquivos de treino correspondentes aos ataques de varredura de portas e de negação de serviço por inundação SYN. O resultado elevou a revocação para 0,9976 e o F1 para 0,9986, ao custo de um aumento modesto nos falsos positivos sobre o tráfego benigno, de 2 para 151 fluxos. Esse contraste sustenta a hipótese levantada: a baixa detecção observada no experimento original decorria majoritariamente da inadequação do conjunto de regras ao tipo de tráfego avaliado, e não de uma limitação do motor de detecção do Suricata.

Essa constatação tem implicações diretas para o objetivo mais amplo deste trabalho. Ao esclarecer a origem do desempenho destoante do Suricata, a investigação remove um obstáculo metodológico importante para que a comparação entre essa ferramenta e o Oráculo seja conduzida de forma mais criteriosa, com ambos os sistemas avaliados sobre bases de contagem coerentes com seus próprios critérios de operação.

REFERÊNCIAS

- 1 MAGLARAS, L.; KANTZAVELOU, I.; FERRAG, M. A. Digital transformation and cybersecurity of critical infrastructures. *Applied Sciences*, v. 11, n. 18, 2021. ISSN 2076-3417. Disponível em: <<https://www.mdpi.com/2076-3417/11/18/8357>>.
- 2 REHAN, H. *Deep Learning for Network Traffic Analysis: Detecting Security Breaches in Real-Time*. 2022. <https://www.researchgate.net/publication/390465875_Deep_Learning_for_Network_Traffic_Analysis_Detecting_Security_Breaches_in_Real-Time>. Available on ResearchGate. Disponível em: <https://www.researchgate.net/publication/390465875_Deep_Learning_for_Network_Traffic_Analysis_Detecting_Security_Breaches_in_Real-Time>.
- 3 WU, Y.; ZOU, B.; CAO, Y. Current status and challenges and future trends of deep learning-based intrusion detection models. *Journal of Imaging*, v. 10, n. 10, 2024. ISSN 2313-433X. Disponível em: <<https://www.mdpi.com/2313-433X/10/10/254>>.
- 4 GOMES, J. E. C.; EHLERT, R. R.; BOESCHE, R. M.; LIMA, V. S. d.; STOCCHERO, J. M.; BARONE, D. A. C.; WICKBOLDT, J. A.; FREITAS, E. Pignaton de; ANJOS, J. C. S. D.; FERNANDES, R. Q. d. A. Surveying emerging network approaches for military command and control systems. *ACM Computing Surveys*, v. 56, n. 6, p. 1–38, 2024. Disponível em: <<https://doi.org/10.1145/3626090>>.
- 5 BRASIL. *Módulo de Proteção Cibernética*. 2025. Exército Brasileiro. Disponível em: <http://mpc.eb.mil.br/_static/mpc-docs.pdf>. Acesso em: 14 mar 2026.
- 6 LIMA, M. G.; CARVALHO, A.; ÁLVARES, J. G.; CHAGAS, C. E. das; GOLDSCHMIDT, R. R. *Impacts of Data Preprocessing and Hyperparameter Optimization on the Performance of Machine Learning Models Applied to Intrusion Detection Systems*. 2024. Disponível em: <<https://arxiv.org/abs/2407.11105>>.
- 7 SANTOS, F. L. dos; CUADROS, D. B. Trabalho acadêmico, *Implementação de uma camada de proteção baseada em anomalias no Sistema de Detecção de Intrusão do Módulo de Proteção Cibernética utilizando técnicas de Aprendizado de Máquina*. Rio de Janeiro: [s.n.], 2024. 76 f. Projeto Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia da Computação.
- 8 ASSUNÇÃO, F. A. R.; SOARES, R. R. C. Trabalho acadêmico, *Implementação de uma camada de proteção baseada em anomalias no Sistema de Detecção de Intrusão (IDS) do Módulo de Proteção Cibernética utilizando técnicas de Aprendizagem de Máquina*. Rio de Janeiro: [s.n.], 2025. Projeto Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia da Computação.
- 9 FUENTES-GARCÍA, N. M.; CAMACHO, J.; MACÍÁ-FERNÁNDEZ, G. Present and future of network security monitoring. *IEEE Access*, PP, p. 1–1, 03 2021.
- 10 KHRAISAT, A.; ALAZAB, M. A critical review of intrusion detection systems in the internet of things: techniques, deployment strategy, validation strategy, attacks, public datasets and challenges. *Cybersecurity*, Springer, v. 4, n. 1, p. 1–27, 2021. Disponível em: <<https://link.springer.com/article/10.1186/s42400-021-00077-7>>.

- 11 MARTINS, I.; RESENDE, J.; SOUSA, P.; SILVA, S. Host-based ids: A review and open issues of an anomaly detection system in iot. *Journal of Systems Architecture*, Elsevier, v. 127, p. 102473, 2022. Disponível em: <https://jresende.github.io/paper/Host_based_IDS_a_review_and_open_issues_of_an_anomaly_detection_system_in_IoT.pdf>.
- 12 TYMOSHCHUK, V.; VORONA, M.; DOLINSKYI, A.; SHYMANSKA, V.; TYMOSHCHUK, D. Security onion platform as a tool for detecting and analysing cyber threats. *Collection of scientific papers*, n. December 13, 2024; Zurich, Switzerland, p. 232–237, Jan. 2025. Disponível em: <<https://archive.logos-science.com/index.php/conference-proceedings/article/view/2538>>.
- 13 Netgate. *pfSense Documentation*. <<https://docs.netgate.com/pfsense/>>.
- 14 Proxmox Server Solutions GmbH. *Proxmox Virtual Environment*. <<https://pve.proxmox.com/>>.
- 15 GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016. Disponível em: <<https://www.deeplearningbook.org>>.
- 16 HORNIK, K. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, v. 4, n. 2, p. 251–257, 1991. Disponível em: <[https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T)>.
- 17 GLOT, X.; BENGIO, Y. Understanding the difficulty of training deep feedforward neural networks. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. JMLR Workshop and Conference Proceedings, 2010. v. 9, p. 249–256. Disponível em: <<https://proceedings.mlr.press/v9/glot10a.html>>.
- 18 HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015. Disponível em: <<https://arxiv.org/abs/1512.03385>>.
- 19 HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Disponível em: <<https://doi.org/10.1162/neco.1997.9.8.1735>>.
- 20 VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>.
- 21 JACOBS, R. A.; JORDAN, M. I.; NOWLAN, S. J.; HINTON, G. E. Adaptive mixtures of local experts. *Neural Computation*, v. 3, n. 1, p. 79–87, 1991. Disponível em: <<https://doi.org/10.1162/neco.1991.3.1.79>>.
- 22 SHAZEER, N.; MIRHOSEINI, A.; MAZIARZ, K. et al. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2017. Disponível em: <<https://arxiv.org/abs/1701.06538>>.
- 23 Canadian Institute for Cybersecurity. *CICFlowMeter*. <<https://www.unb.ca/cic/research/applications.html>>. Acesso em: [data].

- 24 RODRÍGUEZ, M.; ALESANCO, ; MEHAVILLA, L.; GARCÍA, J. Evaluation of machine learning techniques for traffic flow-based intrusion detection. *Sensors*, v. 22, n. 23, p. 9326, 2022.
- 25 Broadcom Inc. *RabbitMQ Documentation*. <<https://www.rabbitmq.com/docs>>. Acesso em: [data].
- 26 Prometheus Authors. *Prometheus Documentation*. <<https://prometheus.io/docs>>. Acesso em: [data].
- 27 Prometheus Authors. *Prometheus Pushgateway*. <<https://github.com/prometheus/pushgateway>>. Acesso em: [data].
- 28 Elasticsearch B.V. *The Elastic Stack*. <<https://www.elastic.co/docs/get-started/the-stack>>. Acesso em: [data].
- 29 Open Information Security Foundation (OISF). *Suricata User Guide*. <<https://docs.suricata.io/>>. Acesso em: [data].
- 30 S. Dadkhah and E. C. P. Neto and R. Ferreira and R. C. Molokwu and S. Sadeghi and A. A. Ghorbani. *Attack Vectors in Healthcare Devices: A Multi-Protocol Dataset for Assessing IoMT Device Security (CIC-IoMT2024)*. 2024. Disponível em: <<https://www.unb.ca/cic/datasets/iomt-dataset-2024.html>>.
- 31 Proofpoint, Inc. *ET Open Rules*. <https://rules.emergingthreats.net/OPEN_download_instructions.html>. Acesso em: [data].